# Tina4 for Node.js Developers

This is not a framework

Andre van Zuydam

v3.1.1 - 2026

# Table of Contents

1. Chapter 1: Getting Started with Tina4 Node.js
2. Chapter 2: Routing
3. Chapter 3: Request & Response
4. Chapter 4: Templates
5. Chapter 5: Database
6. Chapter 6: ORM
7. Chapter 7: Authentication
8. Chapter 8: Middleware
9. Chapter 9: Sessions & Cookies
10. Chapter 10: API Documentation with Swagger
11. Chapter 11: Queue System
12. Chapter 12: Real-time with WebSocket
13. Chapter 13: Email with Messenger
14. Chapter 14: Caching
15. Chapter 15: Frontend Integration
16. Chapter 16: GraphQL
17. Chapter 17: Testing
18. Chapter 18: Dev Tools
19. Chapter 19: CLI & Scaffolding
20. Chapter 20: Deployment
21. Chapter 21: Building a Complete App
22. Chapter 22: Vibe Coding with AI

# Chapter 1: Getting Started with Tina4 Node.js

## 1. What Is Tina4 Node.js

Tina4 Node.js is a zero-dependency web framework. One npm package. Under 5,000 lines of code. It hands you routing, an ORM, a template engine, authentication, queues, WebSocket, and 70 other features. Node.js 20+ and TypeScript.

It belongs to the Tina4 family -- four identical frameworks in Python, PHP, Ruby, and Node.js. Learn one, know all four. Same project structure. Same template syntax. Same CLI commands. Same `.env` variables.

Tina4 Node.js uses `camelCase` for method names (`fetchOne()`, `softDelete()`, `hasMany()`). JavaScript convention. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a working project with an API endpoint and a rendered HTML page.

## 2. Prerequisites and Installation

### *What You Need*

1. **Node.js 20 or later** -- check with:

```
node -v
```

You should see output like:

```
v20.11.1
```

Anything below 20 means you need to upgrade first.

2. **npm** -- Node's package manager. Check with:

```
npm -v
```

You should see:

```
10.2.4
```

npm ships with Node.js. If Node.js is installed, npm is too.

3. **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

```
# macOS (Homebrew)
brew install tina4stack/tap/tina4

# Linux / macOS (install script)
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash

# Windows (PowerShell)
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI:

```
tina4 --version
```

```
tina4 0.1.0
```

4. **TypeScript and tsx** -- for running TypeScript directly:

```
npm install -g tsx
```

Verify:

```
tsx --version
```

### *Creating a New Project*

The Tina4 CLI scaffolds a new project in one command:

```
tina4 init my-store
```

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: Node.js (package.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
// ...
```

Install the Node.js dependencies:

```
cd my-store
npm install
```

```
added 1 package in 2s

1 package installed
```

One package. No dependency tree. No version conflicts. Just `tina4-nodejs`.

### *Starting the Dev Server*

```
tina4 serve

 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 Node.js v3.0.0
  Server running at http://0.0.0.0:7148
```

```
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7148`. The Tina4 welcome page greets you.

Hit the health endpoint:

```
curl http://localhost:7148/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.0.0",
  "framework": "tina4-nodejs"
}
```

Your project is alive.

## 3. Project Structure Walkthrough

Here is what `tina4 init` built:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
■■■ package.json            # npm package definition
■■■ package-lock.json       # Locked dependency versions
■■■ tsconfig.json           # TypeScript configuration
■■■ app.ts                  # Application entry point
■■■ node_modules/           # npm packages (gitignored)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ migrations/         # SQL migration files
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
// ...
```

**Key directories:**

• **`src/routes/`** -- Every `.ts` file here is auto-loaded at startup. Drop your route definitions here. Organize into subdirectories if you want. Tina4 also supports file-based routing: a file at `src/routes/api/users/get.ts` maps to `GET /api/users` with zero configuration.

• **`src/orm/`** -- Every `.ts` file here is auto-loaded. ORM model classes live here.

• **`src/templates/`** -- Frond looks here when you call `res.html()` with a template.

• **`src/public/`** -- Files served directly. `src/public/images/logo.png` becomes `/images/logo.png`.

• **`data/`** -- The default SQLite database (`app.db`) lives here. Gitignored because databases do not belong in version control.

## 4. Your First Route

Time to build an API endpoint that returns a JSON greeting.

### *Explicit Route Registration*

Create the file `src/routes/greeting.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greeting/:name", async (req, res) => {
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
});
```

Save the file. The dev server picks up the change. If live reload is off, restart with `tina4 serve`.

### *File-Based Routing Alternative*

Create the file `src/routes/api/greeting/[name]/get.ts`:

```
export default async (req, res) => {
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
};
```

Both approaches produce identical results. File-based routing maps the file path to the URL. Dynamic segments go in brackets (`[name]`).

### *Test It*

Open your browser to:

```
http://localhost:7148/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000Z"
}
```

Or use curl:

```
curl http://localhost:7148/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00.000Z"}
```

### *Understanding What Happened*

1. You created a file in `src/routes/`. Tina4 discovered it at startup.

2. `Router.get("/api/greeting/:name", ...)` registered a GET route with a path parameter `:name`.

3. A request to `/api/greeting/Alice` hit the router. Pattern matched. Handler fired.

4. `req.params.name` delivered the value `"Alice"` from the URL.

5. `res.json(...)` serialized the object, set `Content-Type: application/json`, and returned `200 OK`.

### *Adding More HTTP Methods*

Add a POST endpoint. Update `src/routes/greeting.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greeting/:name", async (req, res) => {
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
});

Router.post("/api/greeting", async (req, res) => {
    const name = req.body.name ?? "World";
    const language = req.body.language ?? "en";

    const greetings: Record<string, string> = {
        en: "Hello",
        es: "Hola",
        fr: "Bonjour",
        de: "Hallo",
        ja: "Konnichiwa"
    };

    const greeting = greetings[language] ?? greetings["en"];

    return res.status(201).json({
// ...
```

Test the POST endpoint:

```
curl -X POST http://localhost:7148/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (set by `res.status(201)`).

## 5. Your First Template

Tina4 ships with **Frond** -- a zero-dependency, Twig-compatible template engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, Frond will feel familiar.

### *Create a Base Layout*

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
```

```
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0; }
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em; }
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
// ...
```

This base layout defines two blocks (`title` and `content`) that child templates override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

### *Create a Product Listing Page*

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.inStock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
{% endblock %}
```

### *Create the Route That Renders the Template*

Create `src/routes/pages.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/products", async (req, res) => {
    const products = [
        {
            name: "Wireless Keyboard",
```

```
            description: "Ergonomic wireless keyboard with backlit keys.",
            price: 79.99,
            inStock: true
        },
        {
            name: "USB-C Hub",
            description: "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            price: 49.99,
            inStock: true
        },
        {
            name: "Monitor Stand",
            description: "Adjustable aluminum monitor stand with cable management.",
            price: 129.99,
            inStock: false
        },
        {
            name: "Mechanical Mouse",
            description: "High-precision wireless mouse with 16,000 DPI sensor.",
// ...
```

File-based routing works here too. Create `src/routes/products/get.ts`:

```
export const template = "products.html";

export default async (req, res) => {
    return {
        products: [
            { name: "Wireless Keyboard", description: "Ergonomic wireless keyboard.", price: 79.99, inStock: t
            // ... more products
        ]
    };
};
```

Export a `template` constant and Tina4 renders it with the data returned from the handler.

### *See It in the Browser*

Open `http://localhost:7148/products`. You should see:

• A dark navigation bar at the top with "Home" and "Products" links

• The heading "Our Products"

• A subheading showing "Showing 4 products"

• Four product cards, each with a name, description, price, and stock badge

• The "Monitor Stand" card wears a red "Out of Stock" badge

• The other three wear green "In Stock" badges

### *How Template Rendering Works*

1. `res.html("products.html", { products })` tells Frond to render `src/templates/products.html` with the given data.

2. Frond sees `{% extends "base.html" %}` and loads the base template.

3. The `{% block content %}` in `products.html` replaces the same block in `base.html`.

4. `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

5. `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.

6. `{% for product in products %}` loops through the array.

7. `{% if product.inStock %}` conditionally renders the stock badge.

8. `{{ products | length }}` returns the count of items in the array.

### *About tina4css*

The `tina4.css` file in the base template is Tina4's built-in CSS utility framework. Layout utilities, typography, and common UI patterns -- without Bootstrap or Tailwind. Auto-provided at scaffolding time. No separate download needed.

## 6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

| Variable | Default Value | What It Means |

|----------|---------------|---------------|

| `TINA4_PORT` | `7148` | Server runs on port 7148 |

| `DATABASE_URL` | `sqlite:///data/app.db` | SQLite database in the `data/` directory |

| `TINA4_LOG_LEVEL` | `ALL` | All log messages are output |

| `CORS_ORIGINS` | `*` | All origins allowed (fine for development) |

| `TINA4_RATE_LIMIT` | `60` | 60 requests per minute per IP |

To change the port, add it to `.env`:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

For the complete `.env` reference with all 68 variables, see Book 0, Chapter 4.

## 7. The Dev Dashboard

With `TINA4_DEBUG=true`, Tina4 provides a built-in development dashboard. Enable it by adding a console token to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart the server and navigate to:

```
http://localhost:7148/tina4/console
```

Enter your token (`my-dev-token`) when prompted. The dashboard reveals:

• **System Overview** -- framework version, Node.js version, uptime, memory usage, database status

• **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.

• **Error Log** -- unhandled exceptions with stack traces and occurrence counts

• **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)

• **WebSocket Monitor** -- active WebSocket connections with metadata

• **Routes** -- all registered routes with their methods, paths, and middleware

The console shows you what your application is doing without littering your code with `console.log` statements.

When you visit any HTML page (like `/products`), a **debug overlay** appears -- a toolbar at the bottom of the page showing:

• Request details (method, URL, duration)

• Database queries executed (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

This overlay lives only in debug mode. Production never sees it.

## 8. The app.ts Entry Point

The `app.ts` file is the entry point:

```
import { Tina4 } from "tina4-nodejs";

const app = new Tina4();
app.start();
```

That is the entire file. Tina4 discovers routes, models, and templates from the `src/` directory. You register nothing manually.

To start the server without the CLI:

```
npx tsx app.ts

 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 Node.js v3.0.0
  Server running at http://0.0.0.0:7148
```

Both `tina4 serve` and `npx tsx app.ts` work. The CLI adds live reload and other development features.

# 9. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

## *Exercise Part A: Greeting API*

Create an API endpoint at `GET /api/greet` that:

1. Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

2. If `name` is missing, defaults to `"Stranger"`

3. Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

4. The `time_of_day` should be calculated from the server's current hour:

• 5:00 - 11:59 = "morning"

• 12:00 - 16:59 = "afternoon"

• 17:00 - 20:59 = "evening"

• 21:00 - 4:59 = "night"

**Test your endpoint with:**

```
curl "http://localhost:7148/api/greet?name=Sarah"
curl "http://localhost:7148/api/greet"
```

## *Exercise Part B: Product List Page*

Create a page at `GET /store` that:

1. Displays a list of at least 5 products (hardcoded for now)

2. Each product has: name, category, price, and a boolean `featured` flag

3. Featured products should be highlighted (different background color, border, or badge)

4. The page should show the total number of products and the number of featured products

5. Use template inheritance -- create a layout template and a page template that extends it

6. Include `tina4.css` and `frond.js`

**Your products data should look like this in your route handler:**

```
const products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
    { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true },
    { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
];
```

**Expected browser output:**

• A page titled "Our Store"

• Text showing "5 products, 3 featured"

• A list of product cards with name, category, price, and a "Featured" badge on the highlighted items

• Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

## 10. Solutions

### *Solution A: Greeting API*

Create `src/routes/greet.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greet", async (req, res) => {
    const name = req.query.name ?? "Stranger";
    const hour = new Date().getHours();

    let timeOfDay: string;
    if (hour >= 5 && hour < 12) {
        timeOfDay = "morning";
    } else if (hour >= 12 && hour < 17) {
        timeOfDay = "afternoon";
    } else if (hour >= 17 && hour < 21) {
        timeOfDay = "evening";
    } else {
        timeOfDay = "night";
    }

    return res.json({
        greeting: `Welcome, ${name}!`,
        time_of_day: timeOfDay
    });
});
```

**Test:**

```
curl "http://localhost:7148/api/greet?name=Sarah"
```

```
{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}
```

```
curl "http://localhost:7148/api/greet"
```

```
{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

### *Solution B: Product List Page*

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
```

```
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px; font-size: 0
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
// ...
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/store", async (req, res) => {
    const products = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
        { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
        { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true },
        { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
    ];

    const featuredCount = products.filter(p => p.featured).length;

    return res.html("store.html", {
        products,
        featured_count: featuredCount
    });
});
```

**Open `http://localhost:7148/store` in your browser.** You should see:

- A dark header reading "Our Store"

• Text showing "5 products, 3 featured"

• Five product cards in a grid

• Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge

• Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border

• Each card shows the product name, category, and price formatted with two decimal places

## 11. Gotchas

### *1. File not auto-discovered*

**Problem:** You created a route file but nothing happens when you visit the URL.

**Cause:** The file is not in `src/routes/`. It must be inside `src/routes/` (or a subdirectory of it), and the file must end with `.ts`.

**Fix:** Move the file to `src/routes/your-file.ts` and restart the server.

### *2. "Module not found" errors*

**Problem:** `Cannot find module 'tina4-nodejs'` or similar.

**Cause:** Missing npm install or incorrect import path.

**Fix:** Run `npm install` in your project root. Make sure your import uses the exact package name: `import { Router } from "tina4-nodejs"`.

### *3. JSON response shows HTML*

**Problem:** Your JSON endpoint returns HTML instead of JSON.

**Cause:** You returned a string instead of using `res.json()`. A plain string tells Tina4 to treat it as HTML.

**Fix:** Use `res.json(data)` for JSON endpoints. `console.log()` is not a response mechanism.

### *4. Template not found*

**Problem:** `Template "my-page.html" not found` error.

**Cause:** The template file is not in `src/templates/`, or there is a typo in the filename.

**Fix:** Check that the file exists at `src/templates/my-page.html`. The name in `res.html()` is relative to `src/templates/`.

### *5. Port already in use*

**Problem:** `Error: Address already in use (port 7148)`

**Cause:** Another process is occupying port 7148.

**Fix:** Stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

### *6. Changes not reflected*

**Problem:** You edited a file but the browser shows the old version.

**Cause:** Live reload may not be active. Browser caching can serve stale versions.

**Fix:** Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart the dev server with `Ctrl+C` and `tina4 serve`.

### *7. .env not loaded*

**Problem:** Environment variables have no effect.

**Cause:** The `.env` file must be at the project root (same directory as `package.json`).

**Fix:** Move `.env` to the project root.

# Chapter 2: Routing

## 1. How Routing Works in Tina4

Every web application maps URLs to code. A browser requests `/products`. The framework finds the handler for `/products`. Runs it. Sends back the result. That mapping is routing.

In Tina4 Node.js, you define routes in TypeScript files inside `src/routes/`. Every `.ts` file in that directory -- and its subdirectories -- is auto-loaded when the server starts. No registration files. No central config. Drop a file in. It works.

Tina4 supports two routing styles: **explicit registration** with the `Router` class, and **file-based routing** where the file path becomes the URL.

The simplest possible route using explicit registration:

```
import { Router } from "tina4-nodejs";

Router.get("/hello", async (req, res) => {
    return res.json({ message: "Hello, World!" });
});
```

Save that as `src/routes/hello.ts`, start the server with `tina4 serve`, and visit `http://localhost:7148/hello`:

```
{"message":"Hello, World!"}
```

The same thing using file-based routing. Create `src/routes/hello/get.ts`:

```
export default async (req, res) => {
    return res.json({ message: "Hello, World!" });
};
```

Both approaches produce identical results. Use whichever you prefer -- or mix them.

## 2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one has a static method on the `Router` class:

```
import { Router } from "tina4-nodejs";

Router.get("/products", async (req, res) => {
    return res.json({ action: "list all products" });
});

Router.post("/products", async (req, res) => {
    return res.status(201).json({ action: "create a product" });
});

Router.put("/products/:id", async (req, res) => {
    const id = req.params.id;
    return res.json({ action: `replace product ${id}` });
});

Router.patch("/products/:id", async (req, res) => {
    const id = req.params.id;
    return res.json({ action: `update product ${id}` });
});

Router.delete("/products/:id", async (req, res) => {
```

```
    const id = req.params.id;
    return res.json({ action: `delete product ${id}` });
});
```

For file-based routing, the HTTP method is the filename:

```
src/routes/products/get.ts      → GET /products
src/routes/products/post.ts     → POST /products
src/routes/products/[id]/put.ts → PUT /products/:id
```

Test each one:

```
curl http://localhost:7148/products

{"action":"list all products"}

curl -X POST http://localhost:7148/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'

{"action":"create a product"}

curl -X PUT http://localhost:7148/products/42

{"action":"replace product 42"}

curl -X PATCH http://localhost:7148/products/42

{"action":"update product 42"}

curl -X DELETE http://localhost:7148/products/42

{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

## 3. Path Parameters

Path parameters capture values from the URL. Prefix the parameter name with a colon:

```
import { Router } from "tina4-nodejs";

Router.get("/users/:id/posts/:postId", async (req, res) => {
    const userId = req.params.id;
    const postId = req.params.postId;

    return res.json({
        user_id: userId,
        post_id: postId
    });
});
```

For file-based routing, wrap the parameter name in brackets:

```
src/routes/users/[id]/posts/[postId]/get.ts

curl http://localhost:7148/users/5/posts/99

{"user_id":"5","post_id":"99"}
```

Notice `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default.

### *Typed Parameters*

Enforce a type by adding a colon and the type after the parameter name:

```
import { Router } from "tina4-nodejs";

Router.get("/orders/:id:int", async (req, res) => {
    const id = req.params.id; // This is now a number
    return res.json({
        order_id: id,
        type: typeof id
    });
});
```

```
curl http://localhost:7148/orders/42
```

```
{"order_id":42,"type":"number"}
```

Pass a non-integer value and the route refuses to match. You get a 404:

```
curl http://localhost:7148/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

| Type | Matches | Example |

|------|---------|---------|

| `int` | Digits only | `:id:int` matches `42` but not `abc` |

| `float` | Decimal numbers | `:price:float` matches `19.99` |

| `alpha` | Letters only | `:slug:alpha` matches `hello` but not `hello123` |

| `alphanumeric` | Letters and digits | `:code:alphanumeric` matches `abc123` |

## 4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them via `req.query`:

```
import { Router } from "tina4-nodejs";

Router.get("/search", async (req, res) => {
    const q = req.query.q ?? "";
    const page = parseInt(req.query.page ?? "1", 10);
    const limit = parseInt(req.query.limit ?? "10", 10);

    return res.json({
        query: q,
        page,
        limit,
        offset: (page - 1) * limit
    });
});
```

```
curl "http://localhost:7148/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

A missing query parameter yields `undefined` from `req.query.key`. The nullish coalescing operator (`??`) provides defaults.

## 5. Route Groups

A set of routes sharing a common prefix belongs in a group. `Router.group()` eliminates repetition:

```
import { Router } from "tina4-nodejs";

Router.group("/api/v1", () => {

    Router.get("/users", async (req, res) => {
        return res.json({ users: [] });
    });

    Router.get("/users/:id:int", async (req, res) => {
        const id = req.params.id;
        return res.json({ user: { id, name: "Alice" } });
    });

    Router.post("/users", async (req, res) => {
        return res.status(201).json({ created: true });
    });

    Router.get("/products", async (req, res) => {
        return res.json({ products: [] });
    });
});
```

The routes register as `/api/v1/users`, `/api/v1/users/:id`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix.

```
curl http://localhost:7148/api/v1/users
```

```
{"users":[]}
```

Groups nest:

```
import { Router } from "tina4-nodejs";

Router.group("/api", () => {
    Router.group("/v1", () => {
        Router.get("/status", async (req, res) => {
            return res.json({ version: "1.0" });
        });
    });

    Router.group("/v2", () => {
        Router.get("/status", async (req, res) => {
            return res.json({ version: "2.0" });
        });
    });
});
```

```
curl http://localhost:7148/api/v1/status
```

```
{"version":"1.0"}
```

```
curl http://localhost:7148/api/v2/status
```

```
{"version":"2.0"}
```

## 6. Middleware on Routes

Middleware is code that runs before (or after) your route handler. Authentication. Logging. Rate limiting. Input validation. Anything that belongs on multiple routes.

### *Middleware on a Single Route*

Pass middleware as the third argument to any route method:

```
import { Router } from "tina4-nodejs";

async function logRequest(req, res, next) {
    const start = Date.now();
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);

    const result = await next(req, res);

    const duration = Date.now() - start;
    console.log(`  Completed in ${duration}ms`);

    return result;
}

Router.get("/api/data", async (req, res) => {
    return res.json({ data: [1, 2, 3] });
}, "logRequest");
```

The middleware function receives `req`, `res`, and `next`. Call `next(req, res)` to continue to the route handler. Skip the call and the handler never runs -- the chain stops. A locked gate for unauthorized requests.

### *Blocking Middleware*

Middleware that checks for an API key:

```
import { Router } from "tina4-nodejs";

async function requireApiKey(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (apiKey !== "my-secret-key") {
        return res.status(401).json({ error: "Invalid API key" });
    }

    return next(req, res);
}

Router.get("/api/secret", async (req, res) => {
    return res.json({ secret: "The answer is 42" });
}, "requireApiKey");
```

```
curl http://localhost:7148/api/secret
```

```
{"error":"Invalid API key"}
```

```
curl http://localhost:7148/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

### *Middleware on a Group*

Apply middleware to an entire group:

```
import { Router } from "tina4-nodejs";

Router.group("/api/admin", () => {
```

```
    Router.get("/dashboard", async (req, res) => {
        return res.json({ page: "admin dashboard" });
    });

    Router.get("/users", async (req, res) => {
        return res.json({ page: "user management" });
    });

}, "requireAuth");
```

### *Multiple Middleware*

Chain multiple middleware by passing an array:

```
Router.get("/api/important", async (req, res) => {
    return res.json({ data: "important stuff" });
}, ["logRequest", "requireApiKey", "requireAuth"]);
```

Middleware runs in order: `logRequest` first, then `requireApiKey`, then `requireAuth`, then the route handler.

## 7. Route Decorators: @noauth and @secured

Tina4 provides two special decorators for controlling authentication on routes.

### *@noauth -- Public Routes*

When your application has global authentication middleware, `@noauth` marks specific routes as public:

```
import { Router, noauth } from "tina4-nodejs";

/**
 * @noauth
 */
Router.get("/api/public/info", async (req, res) => {
    return res.json({
        app: "My Store",
        version: "1.0.0"
    });
});
```

The `@noauth` decorator tells Tina4 to skip authentication checks for this route, even with global auth middleware configured in `.env`.

### *@secured -- Protected GET Routes*

The `@secured` annotation marks a GET route as requiring authentication:

```
import { Router } from "tina4-nodejs";

/**
 * @secured
 */
Router.get("/api/profile", async (req, res) => {
    return res.json({
        user: req.user
    });
});
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are considered secured. `GET` routes are not -- they are public unless you add `@secured`.

# 8. Wildcard and Catch-All Routes

### *Wildcard Routes*

Use `*` at the end of a path to match anything after it:

```
import { Router } from "tina4-nodejs";

Router.get("/docs/*", async (req, res) => {
    const path = req.params["*"] ?? "";
    return res.json({
        section: "docs",
        path
    });
});
```

```
curl http://localhost:7148/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7148/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

### *Catch-All Route (Custom 404)*

Register a catch-all to handle any unmatched URL:

```
import { Router } from "tina4-nodejs";

Router.get("/*", async (req, res) => {
    return res.status(404).json({
        error: "Page not found",
        path: req.path
    });
});
```

Define this route last. Tina4 matches routes in registration order -- first match wins.

You can also create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for any unmatched route when the file exists.

# 9. Route Listing via CLI

As your application grows, you need visibility into all registered routes. The CLI provides it:

```
tina4 routes

Method   Path                           Middleware          Auth
------   ----                           ----------          ----
```

```
GET      /hello                      -                   public
GET      /products                   -                   public
POST     /products                   -                   secured
PUT      /products/:id               -                   secured
PATCH    /products/:id               -                   secured
DELETE   /products/:id               -                   secured
GET      /api/v1/users               -                   public
GET      /api/v1/users/:id:int       -                   public
POST     /api/v1/users               -                   secured
GET      /api/admin/dashboard        requireAuth         public
GET      /api/admin/users            requireAuth         public
GET      /api/public/info            -                   @noauth
GET      /api/profile                -                   @secured
GET      /search                     -                   public
GET      /docs/*                     -                   public
```

Filter by method or search for a path pattern:

```
tina4 routes --method POST
tina4 routes --filter users
```

## 10. Organizing Route Files

Organize route files however you want. Tina4 loads every `.ts` file in `src/routes/` recursively. Two common patterns:

### *Pattern 1: One File Per Resource*

```
src/routes/
■■■ products.ts     # All product routes
■■■ users.ts        # All user routes
■■■ orders.ts       # All order routes
■■■ pages.ts        # HTML page routes
```

### *Pattern 2: File-Based Routing by Feature*

```
src/routes/
■■■ api/
■   ■■■ products/
■   ■   ■■■ get.ts          # GET /api/products
■   ■   ■■■ post.ts         # POST /api/products
■   ■   ■■■ [id]/
■   ■       ■■■ get.ts      # GET /api/products/:id
■   ■       ■■■ put.ts      # PUT /api/products/:id
■   ■       ■■■ delete.ts   # DELETE /api/products/:id
■   ■■■ users/
■       ■■■ get.ts
■       ■■■ post.ts
■■■ admin/
■   ■■■ dashboard/get.ts
■   ■■■ settings/get.ts
■■■ pages/
    ■■■ home/get.ts
    ■■■ about/get.ts
```

Both patterns work identically. Choose whichever keeps your project navigable.

## 11. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data lives in a TypeScript array (no database yet -- that comes in Chapter 5).

### *Requirements*

Create a file `src/routes/product-api.ts` with the following routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all products. Support `?category=` filter. |

| `GET` | `/api/products/:id:int` | Get a single product by ID. Return 404 if not found. |

| `POST` | `/api/products` | Create a new product. Return 201. |

| `PUT` | `/api/products/:id:int` | Replace a product. Return 404 if not found. |

| `DELETE` | `/api/products/:id:int` | Delete a product. Return 204 with no body. |

Each product has: `id` (number), `name` (string), `category` (string), `price` (number), `inStock` (boolean).

Start with this seed data:

```
let products = [
    { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
    { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
    { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
    { id: 4, name: "Standing Desk", category: "Office", price: 549.99, inStock: true },
    { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true }
];
```

Test with:

```
# List all
curl http://localhost:7148/api/products

# Filter by category
curl "http://localhost:7148/api/products?category=Fitness"

# Get one
curl http://localhost:7148/api/products/3

# Create
curl -X POST http://localhost:7148/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "inStock": true}'

# Update
curl -X PUT http://localhost:7148/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "inStock": true}'

# Delete
curl -X DELETE http://localhost:7148/api/products/3

# Not found
curl http://localhost:7148/api/products/999
```

## 12. Solution

Create `src/routes/product-api.ts`:

```
import { Router } from "tina4-nodejs";

// In-memory product store (resets on server restart)
let products = [
    { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
    { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
    { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
    { id: 4, name: "Standing Desk", category: "Office", price: 549.99, inStock: true },
    { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true }
];

let nextId = 6;

// List all products, optionally filter by category
Router.get("/api/products", async (req, res) => {
    const category = req.query.category ?? null;

    if (category !== null) {
        const filtered = products.filter(
            p => p.category.toLowerCase() === String(category).toLowerCase()
        );
        return res.json({ products: filtered, count: filtered.length });
    }

    return res.json({ products, count: products.length });
// ...
```

**Expected output for the test commands:**

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"inStock":true},{"id":2
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"inStock":true},{"id":5,"name":"Runn
```

Create (Status: `201 Created`):

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"inStock":true}
```

Not found (Status: `404 Not Found`):

```
{"error":"Product not found","id":999}
```

# 13. Gotchas

### *1. Trailing Slashes Matter*

**Problem:** `/products` works but `/products/` returns a 404 (or vice versa).

**Cause:** Tina4 treats `/products` and `/products/` as different routes by default.

**Fix:** Pick one convention and stick with it. Set `TINA4_TRAILING_SLASH_REDIRECT=true` in `.env` and Tina4 redirects `/products/` to `/products`.

### *2. Parameter Names Must Be Unique in a Path*

**Problem:** `/users/:id/posts/:id` gives wrong results -- both parameters share the same name.

**Cause:** The second `:id` overwrites the first in `req.params`.

**Fix:** Use distinct names: `/users/:userId/posts/:postId`.

### *3. Method Conflicts*

**Problem:** You defined `Router.get("/items/:id", ...)` and `Router.get("/items/:action", ...)` and the wrong handler runs.

**Cause:** Both patterns match `/items/42`. The first one registered wins.

**Fix:** Use typed parameters to disambiguate: `Router.get("/items/:id:int", ...)` matches integers only, leaving `/items/export` free for the other route.

### *4. Route Handler Must Return a Response*

**Problem:** Your route handler runs but the browser shows an empty page or a 500 error.

**Cause:** You forgot the `return` statement. Without `return`, the handler produces `undefined` and Tina4 has nothing to send back.

**Fix:** Every handler must `return res.json(...)` or `return res.html(...)`.

### *5. Async Handlers*

**Problem:** Your handler calls an async function but the response returns before it completes.

**Cause:** You forgot to `await` the async operation. The handler returns before the work finishes.

**Fix:** `await` all async operations inside handlers. All Tina4 route handlers should be `async` functions.

### *6. Middleware Function Must Be a Named Function*

**Problem:** Passing an anonymous arrow function as middleware causes an error.

**Cause:** Tina4 expects middleware referenced by function name (a string), not as an inline closure.

**Fix:** Define your middleware as a named function and pass the name as a string: `"myMiddleware"`, not `(req, res, next) => { ... }`.

### *7. Group Prefix Must Start with a Slash*

**Problem:** `Router.group("api/v1", ...)` produces routes that do not match.

**Cause:** The group prefix needs a leading `/`.

**Fix:** Start group prefixes with `/`: `Router.group("/api/v1", ...)`.

# Chapter 3: Request & Response

## 1. The Two Objects You Always Get

Every route handler in Tina4 receives two arguments: `req` and `res`. The request tells you what the client sent. The response is how you talk back. Together they are the entire HTTP conversation.

```
import { Router } from "tina4-nodejs";

Router.get("/echo", async (req, res) => {
    return res.json({
        method: req.method,
        path: req.path,
        your_ip: req.ip
    });
});
```

```
curl http://localhost:7148/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

The pattern for every route: inspect the request, build the response, return it.

## 2. The Request Object

The `req` object gives you everything the client sent. Here is the complete inventory.

### *method*

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
req.method // "GET"
```

### *path*

The URL path without query parameters:

```
// Request to /api/users?page=2
req.path // "/api/users"
```

### *params*

Path parameters from the URL pattern (see Chapter 2):

```
// Route: /users/:id/posts/:postId
// Request: /users/5/posts/42
req.params.id     // "5" (or 5 if typed as :id:int)
req.params.postId // "42"
```

### *query*

Query string parameters as an object:

```
// Request: /search?q=keyboard&page=2&sort=price
req.query.q    // "keyboard"
req.query.page // "2"
req.query.sort // "price"
```

### *body*

The parsed request body. JSON requests produce an object. Form submissions contain form fields:

```
// POST with {"name": "Widget", "price": 9.99}
req.body.name  // "Widget"
req.body.price // 9.99
```

### headers

Request headers as an object. Header names are normalized to lowercase:

```
req.headers["content-type"]  // "application/json"
req.headers["authorization"] // "Bearer eyJhbGci..."
req.headers["x-custom"]      // "my-value"
```

### ip

The client's IP address:

```
req.ip // "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers when behind a reverse proxy.

### cookies

Cookies sent by the client:

```
req.cookies.session_id  // "abc123"
req.cookies.preferences // "dark-mode"
```

### files

Uploaded files (covered in detail in section 7):

```
req.files.avatar // File object with name, type, size, tmpPath
```

### Inspecting the Full Request

A route that dumps everything:

```
import { Router } from "tina4-nodejs";

Router.post("/debug/request", async (req, res) => {
    return res.json({
        method: req.method,
        path: req.path,
        params: req.params,
        query: req.query,
        body: req.body,
        headers: req.headers,
        ip: req.ip,
        cookies: req.cookies
    });
});
```

```
curl -X POST "http://localhost:7148/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "content-type": "application/json",
    "x-custom": "hello",
```

```
    "host": "localhost:7148",
    "user-agent": "curl/8.4.0",
    "accept": "*/*",
    "content-length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

## 3. The Response Object

The `res` object is your toolkit for sending data back to the client. Every method returns the response, so you can chain calls.

### *json() -- JSON Response*

The workhorse for APIs. Pass any object or value and it becomes JSON:

```
return res.json({ name: "Alice", age: 30 });
```

```
{"name":"Alice","age":30}
```

Chain with `status()` for a custom status code:

```
return res.status(201).json({ id: 7, name: "Widget" });
```

This returns `201 Created` with the JSON body.

### *html() -- Template or Raw HTML Response*

Render a Frond template with data:

```
return res.html("products.html", {
    products,
    title: "Our Products"
});
```

Or return raw HTML:

```
return res.html("<h1>Hello</h1><p>This is HTML.</p>");
```

Sets `Content-Type: text/html; charset=utf-8`.

### *text() -- Plain Text Response*

Return plain text:

```
return res.text("Just a plain string.");
```

Sets `Content-Type: text/plain; charset=utf-8`.

### *redirect() -- Redirect Response*

Send the client to a different URL:

```
return res.redirect("/login");
```

This sends a `302 Found` redirect by default. Pass a different status code for permanent redirects:

```
return res.redirect("/new-location", 301);
```

### *file() -- File Download Response*

Send a file to the client for download:

```
return res.file("/path/to/report.pdf");
```

Tina4 sets the appropriate `Content-Type` based on the file extension and adds a `Content-Disposition` header so the browser downloads the file.

Set a custom filename:

```
return res.file("/path/to/report.pdf", "monthly-report-march-2026.pdf");
```

## 4. Status Codes

Every response method chains with `status()`. The most common ones:

| Code | Meaning | When to Use |

|------|---------|-------------|

| `200` | OK | Default. Successful GET, PUT, PATCH. |

| `201` | Created | Successful POST that created a resource. |

| `204` | No Content | Successful DELETE. No body needed. |

| `301` | Moved Permanently | URL has changed forever. |

| `302` | Found | Temporary redirect. |

| `400` | Bad Request | Invalid input from the client. |

| `401` | Unauthorized | Missing or invalid authentication. |

| `403` | Forbidden | Authenticated but not allowed. |

| `404` | Not Found | Resource does not exist. |

| `409` | Conflict | Duplicate or conflicting data. |

| `422` | Unprocessable Entity | Valid JSON but fails business rules. |

| `500` | Internal Server Error | Something broke on the server. |

```
return res.status(201).json({ id: 7, created: true });
```

## 5. Custom Headers

Set response headers with the `header()` method:

```
Router.get("/api/data", async (req, res) => {
    return res
        .header("X-Request-Id", crypto.randomUUID())
        .header("X-Rate-Limit-Remaining", "57")
        .header("Cache-Control", "no-cache")
        .json({ data: [1, 2, 3] });
});
```

### *CORS Headers*

Tina4 handles CORS based on the `CORS_ORIGINS` setting in `.env`. The default `*` allows all origins. For production, lock it down:

```
CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

## 6. Cookies

Set cookies on the response:

```
Router.post("/login", async (req, res) => {
    return res
        .cookie("session_id", "abc123xyz", {
            httpOnly: true,
            secure: true,
            sameSite: "Strict",
            maxAge: 3600,
            path: "/"
        })
        .json({ message: "Logged in" });
});
```

Read cookies from the request:

```
Router.get("/profile", async (req, res) => {
    const sessionId = req.cookies.session_id ?? null;

    if (sessionId === null) {
        return res.status(401).json({ error: "Not logged in" });
    }

    return res.json({ session: sessionId });
});
```

Delete a cookie by setting its `maxAge` to `0`:

```
return res
    .cookie("session_id", "", { maxAge: 0, path: "/" })
    .json({ message: "Logged out" });
```

## 7. File Uploads

Uploaded files arrive via `req.files`. Each file is an object with metadata and a temporary path.

### *Handling a Single File Upload*

```
import { Router } from "tina4-nodejs";

Router.post("/api/upload", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No file uploaded" });
    }

    const file = req.files.image;

    return res.json({
        name: file.name,
        type: file.type,
        size: file.size,
        tmp_path: file.tmpPath
```

```
    });
});
```

```
curl -X POST http://localhost:7148/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "name": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760,
  "tmp_path": "/tmp/tina4_upload_abc123"
}
```

### *Saving the Uploaded File*

```
import { Router } from "tina4-nodejs";
import { rename, mkdir } from "fs/promises";
import { existsSync } from "fs";
import { join, extname } from "path";
import { randomUUID } from "crypto";

Router.post("/api/upload", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No file uploaded" });
    }

    const file = req.files.image;

    // Validate file type
    const allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"];
    if (!allowedTypes.includes(file.type)) {
        return res.status(400).json({ error: "Invalid file type. Allowed: JPEG, PNG, GIF, WebP" });
    }

    // Validate file size (max 5MB)
    const maxSize = 5 * 1024 * 1024;
    if (file.size > maxSize) {
        return res.status(400).json({ error: "File too large. Maximum size: 5MB" });
    }

// ...
```

## 8. File Downloads

Send files to the client using `res.file()`:

```
import { Router } from "tina4-nodejs";
import { existsSync } from "fs";
import { join } from "path";

Router.get("/api/reports/:filename", async (req, res) => {
    const filename = req.params.filename;
    const filepath = join(process.cwd(), "data/reports", filename);

    if (!existsSync(filepath)) {
        return res.status(404).json({ error: "Report not found" });
    }

    return res.file(filepath);
});
```

To force a specific download filename:

```
return res.file(filepath, "Q1-2026-Sales-Report.pdf");
```

## 9. Content Negotiation

Check the `Accept` header to return different formats:

```
import { Router } from "tina4-nodejs";

Router.get("/api/products/:id:int", async (req, res) => {
    const id = req.params.id;
    const product = { id, name: "Wireless Keyboard", price: 79.99 };

    const accept = req.headers["accept"] ?? "application/json";

    if (accept.includes("text/html")) {
        return res.html("product-detail.html", { product });
    }

    if (accept.includes("text/plain")) {
        return res.text(`Product #${id}: ${product.name} - $${product.price}`);
    }

    return res.json(product);
});
```

## 10. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back.

### *Requirements*

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/images` | Upload an image. Validate type and size. Return the image URL. |

| `GET` | `/api/images/:filename` | Return the uploaded image file. Return 404 if not found. |

Rules:

1. Accept JPEG, PNG, and WebP files only

2. Maximum file size: 2MB

3. Save files to `src/public/uploads/` with a unique filename

4. Return the original filename, the saved filename, file size in KB, and the URL

5. The GET endpoint should serve the file directly (not JSON)

## 11. Solution

Create `src/routes/images.ts`:

```
import { Router } from "tina4-nodejs";
import { rename, mkdir } from "fs/promises";
```

```
import { existsSync } from "fs";
import { join, extname } from "path";
import { randomUUID } from "crypto";

Router.post("/api/images", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No image file provided. Use field name 'image'." });
    }

    const file = req.files.image;

    const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (!allowedTypes.includes(file.type)) {
        return res.status(400).json({
            error: "Invalid file type",
            received: file.type,
            allowed: allowedTypes
        });
    }

    const maxSize = 2 * 1024 * 1024;
    if (file.size > maxSize) {
        return res.status(400).json({
// ...
```

**Expected output for upload:**

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_a1b2c3d4-e5f6-7890-abcd-ef1234567890.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_a1b2c3d4-e5f6-7890-abcd-ef1234567890.jpg"
}
```

(Status: `201 Created`)

## 12. Gotchas

### *1. Forgetting `return`*

**Problem:** Your handler runs (log output appears) but the browser shows an empty response or a 500 error.

**Cause:** You wrote `res.json({...})` without `return`.

**Fix:** Write `return res.json({...})`. The response object must be returned from the handler for Tina4 to send it.

### *2. Body Is Undefined for JSON Requests*

**Problem:** `req.body` is `undefined` or empty even though you are sending JSON.

**Cause:** Missing `Content-Type: application/json` header. Without it, Tina4 does not parse the body as JSON.

**Fix:** Include `-H "Content-Type: application/json"` when sending JSON with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` requires `headers: {"Content-Type": "application/json"}`.

### *3. File Uploads Return Empty*

**Problem:** `req.files` is empty even though you are uploading a file.

**Cause:** The form is not using `enctype="multipart/form-data"`, or the curl command uses `-d` instead of `-F`.

**Fix:** For HTML forms, use `<form enctype="multipart/form-data">`. For curl, use `-F "field=@file.jpg"` (with `@`), not `-d`.

### *4. Redirect Loops*

**Problem:** The browser shows "too many redirects" or hangs.

**Cause:** Route A redirects to route B, which redirects back to route A.

**Fix:** Trace the redirect chain in the browser's network inspector. Break the cycle.

### *5. Cookie Not Set*

**Problem:** You called `res.cookie(...)` but the browser does not show the cookie.

**Cause:** `secure: true` means the cookie travels only over HTTPS. Local development uses `http://localhost`. The cookie is dropped.

**Fix:** Set `secure: false` during development.

### *6. Large Request Body Rejected*

**Problem:** POST requests with large bodies return a 413 error.

**Cause:** The request body exceeds the configured maximum size.

**Fix:** Increase `TINA4_MAX_BODY_SIZE` in `.env`. The default is `10mb`.

### *7. Headers Are Lowercase in Node.js*

**Problem:** `req.headers["Content-Type"]` is `undefined` even though the header was sent.

**Cause:** Node.js normalizes all header names to lowercase.

**Fix:** Use lowercase header names: `req.headers["content-type"]`.

# Chapter 4: Templates

## 1. Why Templates

In Chapter 1, you saw `res.html("products.html", data)` produce a full HTML page. That rendering was done by **Frond**, Tina4's built-in template engine. Zero dependencies. Twig-compatible. Built from scratch. If you know Twig, Jinja2, or Nunjucks, you know 90% of Frond.

Templates live in `src/templates/`. Call `res.html("page.html", data)` and Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter covers every feature of the template engine. After this, you build real pages.

## 2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
import { Router } from "tina4-nodejs";

Router.get("/welcome", async (req, res) => {
    return res.html("welcome.html", {
        name: "Alice"
    });
});
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

**Expected browser output:**

```
Hello, Alice!
```

### *Accessing Nested Data*

Dot notation reaches into nested objects:

```
const data = {
    user: {
        name: "Alice",
        email: "alice@example.com",
        address: {
            city: "Cape Town",
            country: "South Africa"
        }
    }
};

return res.html("profile.html", data);
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

**Output:**

```
Alice lives in Cape Town, South Africa.
```

### *Expressions*

Basic expressions work inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

## 3. Template Inheritance

Template inheritance is the engine's most powerful feature. Define a base layout once. Extend it in every page.

### *Base Layout*

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
// ...
```

### *Child Template*

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}

{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
```

```
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
import { Router } from "tina4-nodejs";

Router.get("/about", async (req, res) => {
    return res.html("about.twig", {
        founded_year: 2020,
        team_size: 12,
        office_count: 3
    });
});
```

### *Using `{{ parent() }}`*

Add to a block rather than replace it:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

## 4. Includes

Break templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.inStock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}
```

```
{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

### *Passing Variables to Includes*

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

## 5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

### *The `loop` Variable*

Inside a for loop, Frond provides a special `loop` variable:

| Property | Type | Description |

|----------|------|-------------|

| `loop.index` | int | Current iteration (1-based) |

| `loop.index0` | int | Current iteration (0-based) |

| `loop.first` | bool | True on the first iteration |

| `loop.last` | bool | True on the last iteration |

| `loop.length` | int | Total number of items |

| `loop.revindex` | int | Iterations remaining (1-based) |

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

### *Empty Lists*

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

### *Looping Over Key-Value Pairs*

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

## 6. Conditionals

### *if / elseif / else*

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

### *Ternary Operator*

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

### *Testing for Existence*

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

## 7. Filters

Filters transform values. Apply them with the `|` (pipe) character.

### *Text Filters*

| Filter | Input | Output | Description |

|--------|-------|--------|----------|

| `upper` | `"hello"` | `"HELLO"` | Uppercase |

| `lower` | `"HELLO"` | `"hello"` | Lowercase |

| `capitalize` | `"hello world"` | `"Hello world"` | Capitalize first letter |

| `title` | `"hello world"` | `"Hello World"` | Capitalize each word |

| `trim` | `" hello "` | `"hello"` | Remove whitespace |

| `striptags` | `"<b>bold</b>"` | `"bold"` | Remove HTML tags |

### Number Filters

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `number_format(2)` | `1234.5` | `"1,234.50"` | Format number |

| `round` | `3.7` | `4` | Round to nearest integer |

| `abs` | `-5` | `5` | Absolute value |

### Array Filters

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `length` | `[1,2,3]` | `3` | Count items |

| `join(", ")` | `["a","b","c"]` | `"a, b, c"` | Join with separator |

| `first` | `[1,2,3]` | `1` | First item |

| `last` | `[1,2,3]` | `3` | Last item |

| `reverse` | `[1,2,3]` | `[3,2,1]` | Reverse order |

| `sort` | `[3,1,2]` | `[1,2,3]` | Sort ascending |

### The `default` Filter

```
<p>{{ subtitle | default("No subtitle") }}</p>
<p>{{ user.nickname | default(user.name) | default("Anonymous") }}</p>
```

### The `escape` and `raw` Filters

All `{{ }}` output is auto-escaped for HTML safety. If you trust the content and need raw HTML:

```
{{ trusted_html | raw }}
```

Use `raw` with caution. Apply it only to content you control. Never to user input.

### Chaining Filters

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

## 8. Macros

Macros are reusable template functions. Define once. Call many times.

### Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
```

```
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ value | defaul
    </div>
{% endmacro %}
```

### *Using Macros*

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

## 9. Special Tags

### *{% raw %} -- Literal Output*

When you need to output literal `{{ }}` (for a Vue.js template, for example):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

### *Comments*

```
{# This comment will not appear in the HTML output #}
```

## 10. Template Route Export Pattern

Tina4 Node.js has a special pattern for file-based routes with templates. Export a `template` constant and return data from the handler:

Create `src/routes/catalog/get.ts`:

```
export const template = "catalog.twig";

export default async (req, res) => {
```

```
    const category = req.query.category ?? "";

    const products = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
        { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, featured: true }
    ];

    const filtered = category
        ? products.filter(p => p.category.toLowerCase() === category.toLowerCase())
        : products;

    return {
        products: filtered,
        active_category: category,
        categories: [...new Set(products.map(p => p.category))]
    };
};
```

Tina4 renders `src/templates/catalog.twig` with the returned data. The route handler stays clean -- it returns data, the framework handles rendering.

## 11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

### *Requirements*

1. Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`

2. Create a macro file at `src/templates/catalog-macros.twig` with a `productCard(product)` macro and a `categoryFilter(categories, active)` macro

3. Create a page template at `src/templates/catalog.twig` that extends the base, uses the macros, shows category filter buttons, and shows product cards in a grid

4. Create a route at `GET /catalog` that accepts an optional `?category=` filter

### *Data*

```
const products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, inStock: true, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, inStock: true, featured: true },
    { name: "Blender", category: "Kitchen", price: 89.99, inStock: false, featured: false },
    { name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true, featured: false },
    { name: "Desk Lamp", category: "Office", price: 39.99, inStock: true, featured: true },
    { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, inStock: true, featured: false }
];
```

## 12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; back
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radius: 20px; text
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 16px
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge-featured { background: #f39c12; color: white; display: inline-block; padding: 2px 8px; border-r
        .badge-stock { background: #d4edda; color: #155724; display: inline-block; padding: 2px 8px; border-ra
        .badge-nostock { background: #f8d7da; color: #721c24; display: inline-block; padding: 2px 8px; border-
        .empty-state { text-align: center; padding: 40px; color: #6c757d; }
// ...
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.inStock %}
                <span class="badge-stock">In Stock</span>
            {% else %}
                <span class="badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active' : '' }}">All<
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active' : '' }}">{{ ca
// ...
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
```

```
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_category %} in 
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
// ...
```

Create `src/routes/catalog.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/catalog", async (req, res) => {
    const allProducts = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, inStock: true, featured: true },
        { name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, inStock: true, featured: true },
        { name: "Blender", category: "Kitchen", price: 89.99, inStock: false, featured: false },
        { name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true, featured: false },
        { name: "Desk Lamp", category: "Office", price: 39.99, inStock: true, featured: true },
        { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, inStock: true, featured: false }
    ];

    const categories = [...new Set(allProducts.map(p => p.category))].sort();
    const activeCategory = req.query.category ?? "";

    const products = activeCategory
        ? allProducts.filter(p => p.category.toLowerCase() === String(activeCategory).toLowerCase())
        : allProducts;

    return res.html("catalog.twig", {
        products,
        categories,
        active_category: activeCategory
    });
// ...
```

**Expected browser output for `/catalog`:**

• A dark header with "Product Catalog" and "7 products"

• Filter buttons: All (active), Fitness, Kitchen, Office

• A grid of 7 product cards

• Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge

**Expected browser output for `/catalog?category=Kitchen`:**

• Header shows "3 products in Kitchen"

• The Kitchen filter button is active

• Three cards: Espresso Machine, Blender, Cast Iron Skillet

## 13. Gotchas

### *1. `{% extends %}` Must Be the First Tag*

**Problem:** Template inheritance does not work. The page renders without the base layout.

**Cause:** `{% extends "base.twig" %}` must be the first tag in the template. No exceptions.

**Fix:** Make `{% extends %}` the absolute first thing in the file.

### *2. Undefined Variables Show Nothing*

**Problem:** `{{ username }}` renders as empty instead of showing an error.

**Cause:** Frond outputs nothing for undefined variables. By design.

**Fix:** Use the `default` filter: `{{ username | default("Guest") }}`.

### *3. Auto-Escaping Prevents HTML Output*

**Problem:** You pass HTML content but it appears as literal text.

**Cause:** Auto-escaping converts `<` to `<` for security.

**Fix:** For trusted content, use `{{ content | raw }}`. Never use `raw` on user-supplied input.

### *4. Variable Scope in Includes*

**Problem:** A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

**Cause:** Loop variables are scoped to the loop.

**Fix:** Use `{% set %}` before the loop to accumulate values.

### *5. Macro Arguments Are Positional*

**Problem:** Calling `{{ button("Click", style="danger") }}` does not work.

**Cause:** Frond macros use positional arguments, not keyword arguments.

**Fix:** Pass arguments in the order defined: `{{ button("Click", "/url", "danger") }}`.

### *6. Template File Extension Does Not Matter*

**Problem:** Not sure whether to use `.html`, `.twig`, or `.tpl`.

**Cause:** Frond does not care about the file extension. It processes any file in `src/templates/`.

**Fix:** Pick one extension. Be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML files.

### *7. Filters Are Not JavaScript Functions*

**Problem:** You try `{{ items | count }}` or `{{ name | toUpperCase }}` and get an error.

**Cause:** Frond filters follow Twig conventions, not JavaScript conventions.

**Fix:** Use `{{ items | length }}` instead of `count`. Use `{{ name | upper }}` instead of `toUpperCase`.

# Chapter 5: Database

## 1. From Arrays to Real Data

In Chapters 2 and 3, all your data lived in TypeScript arrays. Server restart. Data gone. That works for learning routing and responses. Real applications need persistent storage.

This chapter covers Tina4's database layer: raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 speaks to five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is identical across all of them. One line in `.env` switches the engine.

## 2. Connecting to a Database

### *The Default: SQLite*

When you scaffold a project with `tina4 init`, Tina4 drops a SQLite database at `data/app.db`. The default `.env` includes:

```
TINA4_DEBUG=true
```

With no explicit `DATABASE_URL`, Tina4 defaults to `sqlite:///data/app.db`. That is why the health check at `/health` shows `"database": "connected"` with zero configuration.

### *Connection Strings for Other Databases*

Set `DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

### *Separate Credentials*

```
DATABASE_URL=postgres://localhost:5432/myapp
DATABASE_USERNAME=myuser
DATABASE_PASSWORD=secretpassword
```

### *Verifying the Connection*

```
curl http://localhost:7148/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
  "version": "3.0.0",
```

```
  "framework": "tina4-nodejs"
}
```

## 3. Getting the Database Object

In your route handlers, access the database via the `Database` class:

```
import { Router, Database } from "tina4-nodejs";

Router.get("/api/test-db", async (req, res) => {
    const db = Database.getConnection();

    const result = await db.fetch("SELECT 1 + 1 AS answer");

    return res.json(result);
});
```

```
curl http://localhost:7148/api/test-db
```

```
{"answer": 2}
```

`Database.getConnection()` returns the active database connection. Call `fetch()`, `execute()`, and `fetchOne()` on this object. All database methods are async. All return Promises.

## 4. Raw Queries

### *fetch() -- Get Multiple Rows*

```
const db = Database.getConnection();

const products = await db.fetch("SELECT * FROM products WHERE price > 50");
```

Each row is a plain object with column names as keys:

```
// products looks like:
[
    { id: 1, name: "Keyboard", price: 79.99 },
    { id: 4, name: "Standing Desk", price: 549.99 }
]
```

### *fetchOne() -- Get a Single Row*

```
const product = await db.fetchOne("SELECT * FROM products WHERE id = 1");
// Returns: { id: 1, name: "Keyboard", price: 79.99 }
```

If no row matches, `fetchOne()` returns `null`.

### *execute() -- Run a Statement*

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
await db.execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)");
await db.execute("UPDATE products SET price = 89.99 WHERE id = 1");
await db.execute("DELETE FROM products WHERE id = 5");
await db.execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at TEXT)");
```

### *Full Example: A Simple Query Route*

```
import { Router, Database } from "tina4-nodejs";

Router.get("/api/products", async (req, res) => {
```

```
    const db = Database.getConnection();

    const products = await db.fetch("SELECT * FROM products ORDER BY name");

    return res.json({
        products,
        count: products.length
    });
});
```

## 5. Parameterised Queries

Never concatenate user input into SQL strings. That door leads to SQL injection:

```
// NEVER do this:
await db.fetch(`SELECT * FROM products WHERE name = '${userInput}'`);
```

Instead, use parameterised queries:

```
const db = Database.getConnection();

// Named parameters
const product = await db.fetchOne(
    "SELECT * FROM products WHERE id = :id",
    { id: 42 }
);

// Positional parameters
const products = await db.fetch(
    "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
    [10.00, 100.00]
);
```

### *A Safe Search Endpoint*

```
import { Router, Database } from "tina4-nodejs";

Router.get("/api/products/search", async (req, res) => {
    const db = Database.getConnection();

    const q = req.query.q ?? "";
    const maxPrice = parseFloat(req.query.max_price ?? "99999");

    if (!q) {
        return res.status(400).json({ error: "Query parameter 'q' is required" });
    }

    const products = await db.fetch(
        "SELECT * FROM products WHERE name LIKE :query AND price <= :maxPrice ORDER BY name",
        { query: `%${q}%`, maxPrice }
    );

    return res.json({
        query: q,
        max_price: maxPrice,
        results: products,
        count: products.length
    });
});
```

```
curl "http://localhost:7148/api/products/search?q=key&max_price=100"
```

```
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

## 6. Transactions

When you need multiple operations to succeed or fail together, use transactions:

```
import { Router, Database } from "tina4-nodejs";

Router.post("/api/orders", async (req, res) => {
    const db = Database.getConnection();
    const body = req.body;

    try {
        await db.startTransaction();

        await db.execute(
            "INSERT INTO orders (customer_id, total, status) VALUES (:customerId, :total, 'pending')",
            { customerId: body.customer_id, total: body.total }
        );

        const order = await db.fetchOne("SELECT last_insert_rowid() AS id");
        const orderId = order.id;

        for (const item of body.items) {
            await db.execute(
                "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:orderId, :productId,
                {
                    orderId,
                    productId: item.product_id,
                    qty: item.quantity,
                    price: item.price
// ...
```

## 7. Schema Inspection

### *getTables()*

```
const db = Database.getConnection();
const tables = await db.getTables();
// Returns: ["orders", "order_items", "products", "users"]
```

### *getColumns()*

```
const columns = await db.getColumns("products");
// Returns: [
//     { name: "id", type: "INTEGER", nullable: false, primary: true },
//     { name: "name", type: "TEXT", nullable: false, primary: false },
//     ...
// ]
```

### *tableExists()*

```
if (await db.tableExists("products")) {
    // Table exists, safe to query
}
```

## 8. Batch Operations with executeMany()

Insert or update many rows efficiently:

```
const db = Database.getConnection();

const products = [
    { name: "Widget A", price: 9.99 },
    { name: "Widget B", price: 14.99 },
    { name: "Widget C", price: 19.99 },
    { name: "Widget D", price: 24.99 }
];

await db.executeMany(
    "INSERT INTO products (name, price) VALUES (:name, :price)",
    products
);
```

## 9. Helper Methods: insert(), update(), delete()

### *insert()*

```
const db = Database.getConnection();

await db.insert("products", {
    name: "Wireless Mouse",
    price: 34.99,
    in_stock: 1
});

// Insert multiple rows
await db.insert("products", [
    { name: "USB Cable", price: 9.99, in_stock: 1 },
    { name: "HDMI Cable", price: 14.99, in_stock: 1 }
]);
```

### *update()*

```
await db.update("products", { price: 39.99, in_stock: 1 }, "id = :id", { id: 7 });
```

### *delete()*

```
await db.delete("products", "id = :id", { id: 7 });
```

## 10. Migrations

Migrations are versioned SQL scripts. They evolve your database schema over time. Each migration runs once. Never again.

### *Creating a Migration*

```
tina4 migrate:create create_products_table
```

```
Created migration: src/migrations/20260322143000_create_products_table.sql
```

Edit the generated file:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

### *Running Migrations*

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_products_table.sql
Migrations complete. 1 applied.
```

### *Checking Status and Rolling Back*

```
tina4 migrate:status
tina4 migrate:rollback
```

## 11. Query Caching

Enable query caching in `.env`:

```
TINA4_DB_CACHE=true
```

Identical queries with identical parameters return cached results. The cache invalidates itself when you call `execute()`, `insert()`, `update()`, or `delete()` on the same table.

```
// Force a fresh query (bypass cache)
const products = await db.fetch("SELECT * FROM products", [], { noCache: true });

// Clear the entire cache
await db.clearCache();
```

## 12. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

### *Requirements*

1. Create a migration for a `notes` table with: `id`, `title`, `content`, `tag`, `created_at`, `updated_at`

2. Build these API endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/notes` | List all notes. Support `?tag=` and `?search=` filters. |

| `GET` | `/api/notes/:id:int` | Get a single note. 404 if not found. |

| `POST` | `/api/notes` | Create a note. Validate title and content are not empty. |

| `PUT` | `/api/notes/:id:int` | Update a note. 404 if not found. |

| `DELETE` | `/api/notes/:id:int` | Delete a note. 204 on success, 404 if not found. |

## 13. Solution

### *Migration*

Create `src/migrations/20260322143000_create_notes_table.sql`:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS notes;
```

```
tina4 migrate
```

### *Routes*

Create `src/routes/notes.ts`:

```
import { Router, Database } from "tina4-nodejs";

Router.get("/api/notes", async (req, res) => {
    const db = Database.getConnection();
    const tag = req.query.tag ?? "";
    const search = req.query.search ?? "";

    let sql = "SELECT * FROM notes";
    const params: Record<string, any> = {};
    const conditions: string[] = [];

    if (tag) {
        conditions.push("tag = :tag");
        params.tag = tag;
    }

    if (search) {
        conditions.push("(title LIKE :search OR content LIKE :search)");
        params.search = `%${search}%`;
    }

    if (conditions.length > 0) {
        sql += " WHERE " + conditions.join(" AND ");
    }

// ...
```

**Expected output for create (Status: `201 Created`):**

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

# 14. Gotchas

### *1. Forgetting await*

**Problem:** Database operations return `Promise {<pending>}` instead of results.

**Cause:** You forgot to `await` the database call. All Tina4 database methods are async.

**Fix:** Always `await` database calls: `const result = await db.fetch(...)`.

### *2. Connection String Formats*

**Problem:** The database will not connect.

**Cause:** Each database engine expects a specific URL format. A common mistake is omitting the port.

**Fix:** Always include the port. Default ports: PostgreSQL 5432, MySQL 3306, MSSQL 1433, Firebird 3050.

### *3. SQLite File Paths*

**Problem:** SQLite creates a new empty database instead of using the existing one.

**Cause:** Use three slashes for a relative path: `sqlite:///data/app.db`. Four slashes for absolute: `sqlite:////var/data/app.db`.

### *4. Parameterised Queries with LIKE*

**Problem:** `WHERE name LIKE '%:q%'` does not work.

**Cause:** Parameters inside quotes are literal text, not placeholders.

**Fix:** Include the `%` in the parameter value: `{ q: "%" + search + "%" }`. The SQL should be `WHERE name LIKE :q`.

### *5. Boolean Values in SQLite*

**Problem:** SQLite stores booleans as integers (1 and 0).

**Fix:** Cast in your TypeScript code: `inStock: Boolean(row.in_stock)`.

### *6. Migration Already Applied*

**Problem:** You edited a migration file and ran `tina4 migrate` again, but nothing changed.

**Cause:** Once applied, a migration will not run again.

**Fix:** Create a new migration for schema changes. Do not edit applied migrations.

### *7. fetch() Returns Empty Array, Not Null*

**Problem:** You check `if (result === null)` but it never matches when the table is empty.

**Cause:** `fetch()` always returns an array. Only `fetchOne()` returns `null`.

**Fix:** Check with `if (result.length === 0)`.

# Chapter 6: ORM

## 1. From SQL to Objects

In Chapter 5 you wrote raw SQL for every operation. It works. It gets tedious. The same `INSERT INTO products (name, price, ...) VALUES (:name, :price, ...)` pattern over and over. The ORM (Object-Relational Mapper) replaces that repetition with TypeScript classes. Define a class. Map it to a table. Call `save()`, `load()`, and `delete()`.

Tina4's ORM is minimal by design. It does not hide SQL. It gives you convenient methods for common operations and steps aside when you need raw queries.

## 2. Defining a Model

ORM models live in `src/orm/`. Every `.ts` file in that directory is auto-loaded. Same discovery pattern as route files.

Create `src/orm/Product.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";

    id!: number;
    name!: string;
    category: string = "Uncategorized";
    price: number = 0.00;
    inStock: boolean = true;
    createdAt!: string;
    updatedAt!: string;
}
```

That is a complete model. Here is what each piece does:

• **Extends `BaseModel`** -- This gives you `save()`, `load()`, `delete()`, `select()`, and other methods.

• **Public properties** -- Each property maps to a database column. The property name is `camelCase` and the column name is `snake_case`. Tina4 converts between them automatically: `inStock` maps to `in_stock`, `createdAt` maps to `created_at`.

• **`static tableName`** -- The database table this model maps to. If you omit it, Tina4 infers it from the class name: `Product` becomes `products`, `OrderItem` becomes `order_items`.

• **`static primaryKey`** -- The primary key column. Defaults to `"id"`.

• **Default values** -- Properties with defaults (like `category = "Uncategorized"`) are used when creating new records.

## 3. Field Types

Use TypeScript type declarations on your properties:

| TypeScript Type | Database Type (SQLite) | Database Type (PostgreSQL) | Notes |

|----------|----------|----------|----------|

| `number` | INTEGER or REAL | INTEGER or DOUBLE PRECISION | Whole or decimal numbers |

| `string` | TEXT | VARCHAR(255) | Text fields |

| `boolean` | INTEGER | BOOLEAN | SQLite stores as 0/1 |

| `string \| null` | TEXT (nullable) | VARCHAR(255) NULL | Nullable fields |

### *Nullable Fields*

```
description: string | null = null;
discount: number | null = null;
```

## 4. Creating and Saving Records

### *save() -- Insert or Update*

The `save()` method inserts a new record or updates an existing one:

```
import { Router } from "tina4-nodejs";
import { Product } from "../orm/Product";

Router.post("/api/products", async (req, res) => {
    const body = req.body;

    const product = new Product();
    product.name = body.name;
    product.category = body.category ?? "Uncategorized";
    product.price = parseFloat(body.price ?? 0);
    product.inStock = Boolean(body.in_stock ?? true);
    await product.save();

    return res.status(201).json(product.toDict());
});
```

```
curl -X POST http://localhost:7148/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Wireless Keyboard", "category": "Electronics", "price": 79.99}'
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

### *toDict() and toObject()*

`toDict()` returns the model data as a plain object with `snake_case` keys -- matching the database column names. `toObject()` returns `camelCase` keys -- matching the TypeScript property names. Use `toDict()` for API responses. Use `toObject()` for internal TypeScript usage.

### *Updating an Existing Record*

When `id` is already set, `save()` performs an UPDATE:

```
Router.put("/api/products/:id:int", async (req, res) => {
    const product = new Product();
    await product.load(req.params.id);

    if (!product.id) {
        return res.status(404).json({ error: "Product not found" });
    }

    const body = req.body;
    product.name = body.name ?? product.name;
    product.price = parseFloat(body.price ?? product.price);
    product.category = body.category ?? product.category;
    await product.save();

    return res.json(product.toDict());
});
```

## 5. Loading Records

### *load() -- Get by Primary Key*

```
const product = new Product();
await product.load(42);

if (!product.id) {
    // Product with ID 42 not found
}
```

## 6. Deleting Records

```
Router.delete("/api/products/:id:int", async (req, res) => {
    const product = new Product();
    await product.load(req.params.id);

    if (!product.id) {
        return res.status(404).json({ error: "Product not found" });
    }

    await product.delete();

    return res.status(204).json(null);
});
```

## 7. Querying with select()

### *Basic Select*

```
const product = new Product();
const products = await product.select("*");
```

### *Filtering*

```
const product = new Product();

const electronics = await product.select("*", "category = :category", { category: "Electronics" });
```

```
const affordable = await product.select("*", "price < :maxPrice AND in_stock = :inStock", {
    maxPrice: 100,
    inStock: 1
});
```

### *Ordering and Pagination*

```
const product = new Product();
const sorted = await product.select("*", "", {}, "price DESC");

const page = 1;
const perPage = 10;
const offset = (page - 1) * perPage;
const products = await product.select("*", "", {}, "name ASC", perPage, offset);
```

### *A Full List Endpoint with Filters*

```
import { Router } from "tina4-nodejs";
import { Product } from "../orm/Product";

Router.get("/api/products", async (req, res) => {
    const product = new Product();

    const category = req.query.category ?? "";
    const page = parseInt(req.query.page ?? "1", 10);
    const perPage = parseInt(req.query.per_page ?? "20", 10);
    const sort = req.query.sort ?? "name";
    const order = (req.query.order ?? "ASC").toUpperCase();

    const conditions: string[] = [];
    const params: Record<string, any> = {};

    if (category) {
        conditions.push("category = :category");
        params.category = category;
    }

    const filter = conditions.join(" AND ");
    const allowedSorts = ["name", "price", "category", "created_at"];
    const safeSort = allowedSorts.includes(sort) ? sort : "name";
    const safeOrder = order === "DESC" ? "DESC" : "ASC";
    const offset = (page - 1) * perPage;
// ...
```

## 8. Relationships

### *hasMany -- One-to-Many*

A user has many posts. Define the relationship as a static property:

Create `src/orm/User.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class User extends BaseModel {
    static tableName = "users";
    static primaryKey = "id";
    static hasMany = [
        { model: "Post", foreignKey: "user_id" }
    ];

    id!: number;
```

```
    name!: string;
    email!: string;
    createdAt!: string;
}
```

Create `src/orm/Post.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Post extends BaseModel {
    static tableName = "posts";
    static primaryKey = "id";
    static belongsTo = [
        { model: "User", foreignKey: "user_id" }
    ];
    static hasMany = [
        { model: "Comment", foreignKey: "post_id" }
    ];

    id!: number;
    userId!: number;
    title!: string;
    body!: string;
    published: boolean = false;
    createdAt!: string;
}
```

### *Using Relationships*

```
Router.get("/api/users/:id:int", async (req, res) => {
    const user = new User();
    await user.load(req.params.id);

    if (!user.id) {
        return res.status(404).json({ error: "User not found" });
    }

    const posts = await user.posts();

    return res.json({
        user: user.toDict(),
        posts: posts.map(p => p.toDict()),
        post_count: posts.length
    });
});
```

## 9. Eager Loading

The N+1 query problem kills performance. Eager loading stops it cold:

```
const user = new User();
const users = await user.select("*", "", {}, "name ASC", 20, 0, ["posts"]);
```

The seventh argument is an array of relationship names to include. This runs just 2 queries.

### *Nested Eager Loading*

```
const user = new User();
const users = await user.select("*", "", {}, "", 0, 0, ["posts", "posts.comments"]);
```

## 10. Soft Delete

If your model has a `deletedAt` property, Tina4 supports soft delete:

```
import { BaseModel } from "tina4-nodejs";

export class Post extends BaseModel {
    static tableName = "posts";
    static primaryKey = "id";
    static softDelete = true;

    id!: number;
    title!: string;
    body!: string;
    deletedAt: string | null = null;
}
```

With `softDelete = true`:

• `post.delete()` sets `deleted_at` to the current timestamp instead of deleting the row

• `select()` automatically excludes soft-deleted rows

• `post.forceDelete()` permanently removes the row

## 11. Auto-CRUD

Add `static autoCrud = true` and Tina4 generates REST endpoints for you:

```
import { BaseModel } from "tina4-nodejs";

export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";
    static autoCrud = true;

    id!: number;
    name!: string;
    category: string = "Uncategorized";
    price: number = 0.00;
    inStock: boolean = true;
}
```

This automatically registers:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all with pagination |

| `GET` | `/api/products/:id` | Get one by ID |

| `POST` | `/api/products` | Create a new record |

| `PUT` | `/api/products/:id` | Update a record |

| `DELETE` | `/api/products/:id` | Delete a record |

## 12. Exercise: Build a Blog

Build a blog with three models: User, Post, and Comment. Use relationships, eager loading, and auto-CRUD.

### *Requirements*

1. Create three models: User (with autoCrud), Post (belongsTo User, hasMany Comments), Comment (belongsTo Post)

2. Create migrations for all three tables

3. Build custom endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/blog/posts` | List published posts with author info |

| `GET` | `/api/blog/posts/:id:int` | Get a post with author and comments |

| `POST` | `/api/blog/posts/:id:int/comments` | Add a comment to a post |

## 13. Solution

### *Models*

Create `src/orm/User.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class User extends BaseModel {
    static tableName = "users";
    static primaryKey = "id";
    static autoCrud = true;
    static hasMany = [{ model: "Post", foreignKey: "user_id" }];

    id!: number;
    name!: string;
    email!: string;
    createdAt!: string;
}
```

Create `src/orm/Post.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Post extends BaseModel {
    static tableName = "posts";
    static primaryKey = "id";
    static belongsTo = [{ model: "User", foreignKey: "user_id" }];
    static hasMany = [{ model: "Comment", foreignKey: "post_id" }];

    id!: number;
    userId!: number;
    title!: string;
    body!: string;
    published: boolean = false;
    createdAt!: string;
}
```

Create `src/orm/Comment.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Comment extends BaseModel {
    static tableName = "comments";
    static primaryKey = "id";
    static belongsTo = [{ model: "Post", foreignKey: "post_id" }];

    id!: number;
    postId!: number;
    authorName!: string;
    body!: string;
    createdAt!: string;
}
```

### *Routes*

Create `src/routes/blog.ts`:

```
import { Router } from "tina4-nodejs";
import { Post } from "../orm/Post";
import { Comment } from "../orm/Comment";

Router.get("/api/blog/posts", async (req, res) => {
    const post = new Post();
    const posts = await post.select("*", "published = :published", { published: 1 }, "created_at DESC", 0, 0,

    return res.json({
        posts: posts.map(p => p.toDict()),
        count: posts.length
    });
});

Router.get("/api/blog/posts/:id:int", async (req, res) => {
    const post = new Post();
    await post.load(req.params.id);

    if (!post.id) {
        return res.status(404).json({ error: "Post not found" });
    }

    const user = await post.user();
    const comments = await post.comments();

// ...
```

## 14. Gotchas

### *1. Table Naming Convention*

**Problem:** Your model class is `OrderItem` but queries fail because the table does not exist.

**Cause:** Tina4 converts `OrderItem` to `order_items`. If your table is named differently, it will not match.

**Fix:** Set `static tableName = "order_item"` explicitly.

### *2. Relationship Foreign Key Direction*

**Problem:** `hasMany` with the wrong foreign key gives wrong results.

**Cause:** The foreign key is the column on the related table, not the current table.

**Fix:** For `hasMany`, the foreign key is on the child table. For `belongsTo`, it is on the current table.

### *3. camelCase to snake_case Mapping*

**Problem:** Property `userId` does not map to column `user_id`.

**Fix:** Tina4 automatically converts between `camelCase` and `snake_case`. Ensure your database columns use `snake_case`.

### *4. Forgetting save()*

**Problem:** You set properties but the database does not change.

**Fix:** Always call `await product.save()` after modifying properties.

### *5. Forgetting await*

**Problem:** `product.save()` returns a Promise but data is not persisted.

**Fix:** Always `await` ORM methods: `await product.save()`, `await product.load(id)`, `await product.select(...)`.

### *6. select() Returns Model Instances, Not Plain Objects*

**Problem:** You try to use `result.name` and it works, but JSON serialization misses some fields.

**Fix:** Use `result.toDict()` or `result.toObject()` to convert to a plain object for JSON responses.

### *7. Auto-CRUD Endpoint Conflicts*

**Problem:** Your custom route at `/api/products/:id` conflicts with auto-CRUD.

**Fix:** Custom routes defined in `src/routes/` take precedence over auto-CRUD routes.

# Chapter 7: Authentication

## 1. Locking the Door

Every endpoint you have built so far is public. Anyone with the URL can read, create, update, and delete data. Fine for a tutorial. Unacceptable in production.

A real application needs identity. Who is making this request? And are they allowed to make it?

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

## 2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string carrying a payload -- a user ID, a role, whatever you need. The server creates the token at login. The client sends it with every request. The server verifies it without touching the database.

### *Generating a Token*

```
import { Auth } from "tina4-nodejs";

const payload = {
    user_id: 42,
    email: "alice@example.com",
    role: "admin"
};

const token = Auth.getToken(payload);
```

### *Token Expiry*

Configure in `.env`:

```
TINA4_JWT_EXPIRY=3600
```

| Value | Duration |

|-------|----------|

| `900` | 15 minutes |

| `3600` | 1 hour (default) |

| `86400` | 24 hours |

| `604800` | 7 days |

### *Validating a Token*

```
const isValid = Auth.validToken(token);
```

### *Reading the Payload*

```
const payload = Auth.getPayload(token);
// Returns: { user_id: 42, email: "alice@example.com", role: "admin", iat: ..., exp: ... }
```

### *The Secret Key*

Tina4 generates a random key at `secrets/jwt.key` on first run. Or set it yourself:

```
TINA4_JWT_SECRET=my-very-long-and-random-secret-key-at-least-32-chars
```

## 3. Password Hashing

Plain text passwords are a breach waiting to happen.

### *Hashing a Password*

```
import { Auth } from "tina4-nodejs";

const hash = await Auth.hashPassword("my-secure-password");
// Returns: "$2b$10$abc123...long-hash-string..."
```

### *Checking a Password*

```
const isCorrect = await Auth.checkPassword("my-secure-password", storedHash);
```

### *Registration Example*

```
import { Router, Auth, Database } from "tina4-nodejs";

Router.post("/api/register", async (req, res) => {
    const body = req.body;

    if (!body.name || !body.email || !body.password) {
        return res.status(400).json({ error: "Name, email, and password are required" });
    }

    if (body.password.length < 8) {
        return res.status(400).json({ error: "Password must be at least 8 characters" });
    }

    const db = Database.getConnection();

    const existing = await db.fetchOne("SELECT id FROM users WHERE email = :email", { email: body.email });
    if (existing !== null) {
        return res.status(409).json({ error: "Email already registered" });
    }

    const passwordHash = await Auth.hashPassword(body.password);

    await db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        { name: body.name, email: body.email, hash: passwordHash }
// ...
```

## 4. The Login Flow

```
import { Router, Auth, Database } from "tina4-nodejs";

/**
 * @noauth
 */
Router.post("/api/login", async (req, res) => {
    const body = req.body;

    if (!body.email || !body.password) {
```

```
        return res.status(400).json({ error: "Email and password are required" });
    }

    const db = Database.getConnection();

    const user = await db.fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        { email: body.email }
    );

    if (user === null) {
        return res.status(401).json({ error: "Invalid email or password" });
    }

    if (!(await Auth.checkPassword(body.password, user.password_hash))) {
        return res.status(401).json({ error: "Invalid email or password" });
// ...
```

# 5. Protecting Routes

## *Auth Middleware*

```
import { Auth } from "tina4-nodejs";

async function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        return res.status(401).json({ error: "Authorization header required" });
    }

    const token = authHeader.substring(7);

    if (!Auth.validToken(token)) {
        return res.status(401).json({ error: "Invalid or expired token" });
    }

    const payload = Auth.getPayload(token);
    req.user = payload;

    return next(req, res);
}
```

## *Applying Middleware*

```
import { Router } from "tina4-nodejs";

Router.get("/api/profile", async (req, res) => {
    return res.json({
        user_id: req.user.user_id,
        email: req.user.email,
        name: req.user.name
    });
}, "authMiddleware");
```

## *Role-Based Authorization*

```
function requireRole(role: string) {
    return async function (req, res, next) {
        const authHeader = req.headers["authorization"] ?? "";
        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            return res.status(401).json({ error: "Authorization required" });
```

```
        }

        const token = authHeader.substring(7);
        if (!Auth.validToken(token)) {
            return res.status(401).json({ error: "Invalid or expired token" });
        }

        const payload = Auth.getPayload(token);

        if ((payload.role ?? "") !== role) {
            return res.status(403).json({ error: `Forbidden. Required role: ${role}` });
        }

        req.user = payload;
        return next(req, res);
    };
}
```

## 6. CSRF Protection

For form-based applications, include a CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}
    <input type="text" name="name" value="{{ user.name }}">
    <button type="submit">Update Profile</button>
</form>
```

Validate in the handler:

```
Router.post("/profile/update", async (req, res) => {
    if (!Auth.validateFormToken(req.body._token ?? "")) {
        return res.status(403).json({ error: "Invalid form token" });
    }

    return res.redirect("/profile");
});
```

## 7. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

### *Requirements*

| Method | Path | Auth | Description |

|--------|------|------|----------|

| `POST` | `/api/register` | @noauth | Create an account |

| `POST` | `/api/login` | @noauth | Login, return JWT |

| `GET` | `/api/profile` | secured | Get current user profile |

| `PUT` | `/api/profile` | secured | Update name and email |

| `PUT` | `/api/profile/password` | secured | Change password |

## 8. Solution

Create `src/routes/auth.ts`:

```
import { Router, Auth, Database } from "tina4-nodejs";

async function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        return res.status(401).json({ error: "Authorization required. Send: Authorization: Bearer <token>" })
    }
    const token = authHeader.substring(7);
    if (!Auth.validToken(token)) {
        return res.status(401).json({ error: "Invalid or expired token. Please login again." });
    }
    req.user = Auth.getPayload(token);
    return next(req, res);
}

/**
 * @noauth
 */
Router.post("/api/register", async (req, res) => {
    const body = req.body;
    const errors: string[] = [];

    if (!body.name) errors.push("Name is required");
    if (!body.email) errors.push("Email is required");
    if (!body.password) errors.push("Password is required");
// ...
```

## 9. Gotchas

### *1. Token Expiry Confusion*

**Problem:** Tokens that worked yesterday now return 401.

**Fix:** The default lifetime is 1 hour. Issue new tokens at login. Use refresh tokens for long-lived sessions.

### *2. Secret Key Management*

**Problem:** Tokens generated on one server are invalid on another.

**Fix:** Set `TINA4_JWT_SECRET` explicitly in `.env` and use the same value across all servers.

### *3. CORS with Authentication*

**Problem:** Frontend requests with the `Authorization` header fail with a CORS error.

**Fix:** Tina4 handles preflight requests automatically. Make sure `CORS_ORIGINS` is set correctly.

### *4. Storing Tokens in localStorage*

**Problem:** Token stolen via XSS.

**Fix:** Store tokens in `httpOnly` cookies when possible.

### *5. Forgetting @noauth on Login*

**Problem:** Login endpoint returns 401.

**Fix:** Add `@noauth` to login and register routes.

### *6. Password Hash Column Too Short*

**Problem:** Registration fails because the hash is truncated.

**Fix:** Use `TEXT` for the password hash column, not `VARCHAR(50)`.

### *7. Token in URL Query Parameters*

**Problem:** Tokens in URLs leak through browser history and server logs.

**Fix:** Always send tokens in the `Authorization` header, never in the URL.

# Chapter 8: Middleware

## 1. The Gatekeepers

Your API needs CORS headers for the React frontend. Rate limiting for the public endpoints. Auth checking for admin routes. All without cluttering route handlers.

Middleware solves this. Each middleware is a gatekeeper. It does one job and passes control to the next layer. Chapter 2 introduced the concept. This chapter goes deep: built-in middleware, custom middleware, execution order, short-circuiting, and real-world patterns.

## 2. What Middleware Is

A middleware function stands between the incoming HTTP request and your route handler. It receives the request, the response, and a `next` function:

```
async function passthrough(req, res, next) {
    return next(req, res);
}
```

And one that blocks everything:

```
async function blockEverything(req, res, next) {
    return res.status(503).json({ error: "Service unavailable" });
}
```

## 3. Built-in CorsMiddleware

```
CORS_ORIGINS=http://localhost:3000,https://myapp.com
CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
CORS_HEADERS=Content-Type,Authorization,X-API-Key
CORS_MAX_AGE=86400
CORS_CREDENTIALS=true
```

Apply it:

```
import { Router } from "tina4-nodejs";

Router.group("/api", () => {
    Router.get("/products", async (req, res) => {
        return res.json({ products: [] });
    });
}, "CorsMiddleware");
```

## 4. Built-in RateLimiter

```
TINA4_RATE_LIMIT=60
TINA4_RATE_LIMIT_WINDOW=60
```

```
Router.group("/api/public", () => {
    Router.get("/search", async (req, res) => {
        return res.json({ results: [] });
    });
}, "RateLimiter");
```

Custom limits per group:

```
Router.group("/api/public", () => {
    Router.get("/search", async (req, res) => {
        return res.json({ results: [] });
    });
}, "RateLimiter:30");
```

# 5. Writing Custom Middleware

### *Request Logging*

```
async function logRequest(req, res, next) {
    const start = Date.now();
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.path} from ${req.ip}`);

    const result = await next(req, res);

    const duration = Date.now() - start;
    console.log(`  Completed in ${duration}ms`);

    return result;
}
```

### *Request Timing*

```
async function addTiming(req, res, next) {
    const start = Date.now();
    const result = await next(req, res);
    const duration = Date.now() - start;
    res.header("X-Response-Time", `${duration}ms`);
    return result;
}
```

### *IP Whitelist*

```
async function ipWhitelist(req, res, next) {
    const allowedIps = (process.env.ALLOWED_IPS ?? "127.0.0.1").split(",");

    if (!allowedIps.includes(req.ip)) {
        return res.status(403).json({ error: "Access denied", your_ip: req.ip });
    }

    return next(req, res);
}
```

### *Request Validation*

```
async function requireJson(req, res, next) {
    if (["POST", "PUT", "PATCH"].includes(req.method)) {
        const contentType = req.headers["content-type"] ?? "";

        if (!contentType.includes("application/json")) {
            return res.status(415).json({
                error: "Content-Type must be application/json",
                received: contentType
            });
        }
    }

    return next(req, res);
}
```

## 6. Applying Middleware

Single middleware:

```
Router.get("/api/data", async (req, res) => {
    return res.json({ data: [1, 2, 3] });
}, "logRequest");
```

Multiple middleware:

```
Router.post("/api/data", async (req, res) => {
    return res.status(201).json({ created: true });
}, ["logRequest", "requireJson"]);
```

## 7. Route Groups with Shared Middleware

```
Router.group("/api/public", () => {
    Router.get("/products", async (req, res) => {
        return res.json({ products: [] });
    });
    Router.get("/categories", async (req, res) => {
        return res.json({ categories: [] });
    });
}, ["CorsMiddleware", "RateLimiter:30"]);

Router.group("/api/admin", () => {
    Router.get("/users", async (req, res) => {
        return res.json({ users: [] });
    });
}, ["logRequest", "ipWhitelist", "authMiddleware"]);
```

## 8. Middleware Execution Order

Middleware executes from outer to inner:

```
Router.get("/api/test", async (req, res) => {
    console.log("Handler");
    return res.json({ ok: true });
}, ["middlewareA", "middlewareB", "middlewareC"]);
```

Output:

```
A: before
B: before
C: before
Handler
C: after
B: after
A: after
```

Group middleware always runs before route middleware.

## 9. Short-Circuiting

When middleware does not call `next`, the chain dies:

```
async function requireAuth(req, res, next) {
    const token = req.headers["authorization"] ?? "";

    if (!token) {
        return res.status(401).json({ error: "Authentication required" });
    }

    return next(req, res);
}
```

### *Maintenance Mode*

```
async function maintenanceMode(req, res, next) {
    const isMaintenanceMode = process.env.MAINTENANCE_MODE === "true";

    if (isMaintenanceMode) {
        if (req.path === "/health") {
            return next(req, res);
        }
        return res.status(503).json({ error: "Service is undergoing maintenance", retry_after: 300 });
    }

    return next(req, res);
}
```

## 10. Modifying Requests in Middleware

```
async function addRequestId(req, res, next) {
    const { randomUUID } = await import("crypto");
    req.requestId = randomUUID();
    const result = await next(req, res);
    res.header("X-Request-Id", req.requestId);
    return result;
}
```

## 11. Real-World Middleware Stack

```
import { Router } from "tina4-nodejs";

Router.group("/api/v1", () => {
    Router.get("/products", async (req, res) => {
        return res.json({ products: [
            { id: 1, name: "Widget", price: 9.99 },
            { id: 2, name: "Gadget", price: 19.99 }
        ]});
    });

    Router.post("/products", async (req, res) => {
        return res.status(201).json({
            id: 3,
            name: req.body.name ?? "Unknown",
            price: parseFloat(req.body.price ?? 0)
        });
    });
}, ["addRequestId", "logRequest", "CorsMiddleware", "requireApiKey"]);
```

## 12. Exercise: Build an API Key Middleware

Build `validateApiKey` middleware that checks `X-API-Key` header against `API_KEYS` env variable.

### *Requirements*

1. Missing key: return 401 with `{"error": "API key required"}`
2. Invalid key: return 403 with `{"error": "Invalid API key"}`
3. Valid key: attach to `req.apiKey` and continue
4. Apply to a route group with at least two endpoints

## 13. Solution

```
import { Router } from "tina4-nodejs";

async function validateApiKey(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (!apiKey) {
        return res.status(401).json({ error: "API key required" });
    }

    const validKeys = (process.env.API_KEYS ?? "").split(",").map(k => k.trim());

    if (!validKeys.includes(apiKey)) {
        return res.status(403).json({ error: "Invalid API key" });
    }

    req.apiKey = apiKey;
    return next(req, res);
}

Router.group("/api/partner", () => {
    Router.get("/data", async (req, res) => {
        return res.json({
            authenticated_with: req.apiKey,
            data: [{ id: 1, value: "alpha" }, { id: 2, value: "beta" }]
        });
// ...
```

## 14. Gotchas

### *1. Middleware Must Be a Named Function*

**Fix:** Define as a named function and pass the name as a string.

### *2. Forgetting to Return next()*

**Fix:** Always `return await next(req, res)`.

### *3. Middleware Order Matters*

**Fix:** Put `addRequestId` before `logRequest`: `["addRequestId", "logRequest"]`.

### *4. CORS Preflight Returns 404*

**Fix:** Apply `CorsMiddleware` to the group. It handles `OPTIONS` automatically.

### *5. Rate Limiter Counts Preflight Requests*

**Fix:** Put `CorsMiddleware` before `RateLimiter`.

### *6. Middleware File Not Auto-Loaded*

**Fix:** Put middleware functions in a file inside `src/routes/`.

### *7. Short-Circuiting Skips Cleanup*

**Fix:** Put timing/logging middleware at the outermost layer.

# Chapter 9: Sessions & Cookies

## 1. State in a Stateless World

Your e-commerce site needs a shopping cart that persists across page loads. It remembers the user's language preference. It flashes success messages after form submissions. But HTTP has no memory. Every request arrives fresh. No context. No history.

Sessions and cookies give the server memory. They teach it to remember who is making requests and what they have been doing.

## 2. How Sessions Work

A user visits your site for the first time. Tina4 generates a unique session ID, stores it in a cookie on the user's browser, and creates a server-side storage entry keyed by that ID. Every subsequent request carries the cookie. Tina4 looks up the session data and attaches it to `req.session`.

## 3. File Sessions (Default)

Tina4 stores sessions in files out of the box. Zero configuration.

```
import { Router } from "tina4-nodejs";

Router.get("/visit-counter", async (req, res) => {
    const count = (req.session.visit_count ?? 0) + 1;
    req.session.visit_count = count;

    return res.json({
        visit_count: count,
        message: `You have visited this page ${count} time${count === 1 ? "" : "s"}`
    });
});
```

```
curl http://localhost:7148/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

## 4. Redis Sessions

For production deployments with multiple servers:

```
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
TINA4_SESSION_PASSWORD=your-redis-password
```

Your code stays exactly the same. `req.session` works identically.

## 5. MongoDB and Valkey Sessions

```
# MongoDB
```

```
TINA4_SESSION_HANDLER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017

# Valkey
TINA4_SESSION_HANDLER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

## 6. Reading and Writing Session Data

```
import { Router } from "tina4-nodejs";

Router.post("/api/preferences", async (req, res) => {
    const body = req.body;

    req.session.language = body.language ?? "en";
    req.session.theme = body.theme ?? "light";
    req.session.items_per_page = parseInt(body.items_per_page ?? "20", 10);

    return res.json({
        message: "Preferences saved",
        preferences: {
            language: req.session.language,
            theme: req.session.theme,
            items_per_page: req.session.items_per_page
        }
    });
});

Router.get("/api/preferences", async (req, res) => {
    return res.json({
        language: req.session.language ?? "en",
        theme: req.session.theme ?? "light",
        items_per_page: req.session.items_per_page ?? 20
    });
// ...
```

### *Shopping Cart Example*

```
Router.post("/api/cart/add", async (req, res) => {
    const body = req.body;

    if (!req.session.cart) {
        req.session.cart = [];
    }

    const cart = req.session.cart;
    const existingIndex = cart.findIndex(item => item.product_id === parseInt(body.product_id, 10));

    if (existingIndex >= 0) {
        cart[existingIndex].quantity += parseInt(body.quantity ?? "1", 10);
    } else {
        cart.push({
            product_id: parseInt(body.product_id, 10),
            name: body.name,
            price: parseFloat(body.price),
            quantity: parseInt(body.quantity ?? "1", 10)
        });
    }

    req.session.cart = cart;
```

```
    const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);

// ...
```

## 7. Flash Messages

Flash messages exist for exactly one request:

```
Router.post("/profile/update", async (req, res) => {
    // Update profile logic here...

    req.session._flash = {
        type: "success",
        message: "Profile updated successfully"
    };

    return res.redirect("/profile");
});

Router.get("/profile", async (req, res) => {
    const flash = req.session._flash ?? null;
    delete req.session._flash;

    return res.html("profile.html", {
        user: { name: "Alice", email: "alice@example.com" },
        flash
    });
});
```

## 8. Setting and Reading Cookies

```
Router.post("/api/set-language", async (req, res) => {
    const language = req.body.language ?? "en";

    return res
        .cookie("language", language, {
            expires: new Date(Date.now() + 365 * 24 * 60 * 60 * 1000),
            path: "/",
            httpOnly: false,
            secure: false,
            sameSite: "Lax"
        })
        .json({ message: `Language set to ${language}` });
});

Router.get("/api/get-language", async (req, res) => {
    const language = req.cookies.language ?? "en";
    return res.json({ language });
});
```

### *When to Use Cookies vs Sessions*

| Use Cookies For | Use Sessions For |

|----------|----------|

| Language preference | Shopping cart contents |

| Theme preference (light/dark) | User authentication state |

| "Remember this device" flag | Flash messages |

| Non-sensitive, long-lived data | Sensitive, short-lived data |

## 9. Session Security

```
TINA4_SESSION_LIFETIME=3600
TINA4_SESSION_NAME=tina4_session
TINA4_SESSION_SECURE=true
TINA4_SESSION_HTTPONLY=true
TINA4_SESSION_SAMESITE=Lax
```

### *Session Regeneration*

After login, regenerate the session ID to prevent session fixation:

```
Router.post("/login", async (req, res) => {
    // Validate credentials...
    req.sessionRegenerate();
    req.session.user_id = user.id;
    return res.redirect("/dashboard");
});
```

## 10. Exercise: Build a Shopping Cart with Session Storage

Build a shopping cart stored entirely in session data.

### *Requirements*

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/cart/add` | Add an item to the cart |

| `GET` | `/api/cart` | View the cart with totals |

| `PUT` | `/api/cart/:product_id:int` | Update quantity (0 removes) |

| `DELETE` | `/api/cart/:product_id:int` | Remove an item |

| `DELETE` | `/api/cart` | Clear the cart |

## 11. Solution

Create `src/routes/cart.ts`:

```
import { Router } from "tina4-nodejs";

function getCart(session: any): any[] {
    return session.cart ?? [];
}

function cartResponse(cart: any[]) {
    let total = 0;
    let itemCount = 0;
    const items = cart.map(item => {
```

```
        const subtotal = item.price * item.quantity;
        total += subtotal;
        itemCount += item.quantity;
        return { ...item, subtotal: Math.round(subtotal * 100) / 100 };
    });

    return {
        items,
        item_count: itemCount,
        unique_items: cart.length,
        total: Math.round(total * 100) / 100
    };
}

Router.post("/api/cart/add", async (req, res) => {
// ...
```

## 12. Gotchas

### *1. Sessions Do Not Work with curl Without Cookie Flags*

**Fix:** Use `-c cookies.txt -b cookies.txt` with curl.

### *2. Session Data Disappears After Server Restart*

**Fix:** Use Redis for production. Set `TINA4_SESSION_PATH` for persistent file sessions.

### *3. Session Cookie Not Sent Over HTTP in Production*

**Fix:** Ensure `TINA4_SESSION_SECURE` matches your actual protocol.

### *4. Flash Messages Show Twice*

**Fix:** Always clear the flash message immediately after reading: `delete req.session._flash`.

### *5. Large Session Data Causes Slow Requests*

**Fix:** Keep session data small. Store IDs, not entire objects.

### *6. Remember Me Token Not Invalidated on Password Change*

**Fix:** Clear `remember_token` when the password changes.

### *7. Session Fixation*

**Fix:** Call `req.sessionRegenerate()` after successful login.

# Chapter 10: API Documentation with Swagger

## 1. The 47-Endpoint Problem

Your team has 47 API endpoints. The frontend developer asks "what does this endpoint accept?" again. And again.

Swagger kills that question. It generates interactive API documentation from annotations in your route files. The docs stay current because they live in the code.

Tina4 builds a Swagger UI at `/swagger` from doc-block annotations on your routes. No build step. No extra tooling.

## 2. What Swagger/OpenAPI Is

OpenAPI is a specification format for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and renders documentation. Tina4 builds the spec from JSDoc comments in your TypeScript code. No manual spec writing.

## 3. Enabling Swagger

Swagger runs when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7148/swagger
```

The raw spec is at:

```
http://localhost:7148/swagger/json
```

For production:

```
TINA4_SWAGGER=true
```

## 4. Adding Descriptions to Routes

```
import { Router } from "tina4-nodejs";

/**
 * List all products
 * @description Returns a paginated list of all products in the catalog
 * @tags Products
 */
Router.get("/api/products", async (req, res) => {
    return res.json({ products: [] });
});

/**
 * Get a product by ID
 * @description Returns a single product with full details
 * @tags Products
 * @param int id The unique product identifier
 */
Router.get("/api/products/:id:int", async (req, res) => {
    return res.json({ id: req.params.id, name: "Wireless Keyboard", price: 79.99 });
```

```
});

/**
 * Search products
 * @description Search the product catalog by name or category
 * @tags Products
// ...
```

## 5. Documenting Request and Response Schemas

```
/**
 * Create a new product
 * @description Creates a product in the catalog
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float", "inStock": "bool"}
 * @response 201 {"id": "int", "name": "string", "category": "string", "price": "float"}
 * @response 400 {"error": "string"}
 */
Router.post("/api/products", async (req, res) => {
    if (!req.body.name) {
        return res.status(400).json({ error: "Name is required" });
    }
    return res.status(201).json({
        id: 1,
        name: req.body.name,
        category: req.body.category ?? "Uncategorized",
        price: parseFloat(req.body.price ?? 0)
    });
});
```

## 6. Tags for Grouping

```
/**
 * List all users
 * @tags Users
 */
Router.get("/api/users", async (req, res) => {
    return res.json({ users: [] });
});

/**
 * List all orders
 * @tags Orders
 */
Router.get("/api/orders", async (req, res) => {
    return res.json({ orders: [] });
});
```

## 7. Example Values

```
/**
 * Create a new product
 * @tags Products
 * @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99}
 * @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99}
 */
Router.post("/api/products", async (req, res) => {
```

```
    return res.status(201).json({ id: 42, name: req.body.name, price: req.body.price });
});
```

## 8. Try-It-Out from the Swagger UI

The Swagger UI puts a "Try it out" button on every endpoint. Click it and the UI sends a real HTTP request to your running server. Live testing without curl.

## 9. Customizing the Swagger Info Block

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_CONTACT_EMAIL=api@mystore.com
TINA4_SWAGGER_LICENSE=MIT
```

## 10. Generating Client SDKs

```
npm install -g @openapitools/openapi-generator-cli

openapi-generator-cli generate \
  -i http://localhost:7148/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client
```

## 11. Exercise: Document a Complete User API

Document PUT, GET user orders, and POST avatar endpoints with full Swagger annotations including `@param`, `@query`, `@body`, `@response`, and `@example`.

## 12. Solution

```
import { Router } from "tina4-nodejs";

/**
 * Update a user
 * @description Updates an existing user's profile information
 * @tags Users
 * @param int id User ID
 * @body {"name": "string", "email": "string", "role": "string"}
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updated_at": "string"}
 * @response 404 {"error": "string"}
 * @example request {"name": "Alice Smith", "email": "alice.smith@example.com"}
 * @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin", "u
 */
Router.put("/api/users/:id:int", async (req, res) => {
    return res.json({ id: req.params.id, name: req.body.name ?? "Alice", updated_at: new Date().toISOString()
});

/**
 * List user orders
```

```
  * @description Returns a paginated list of orders for a specific user
  * @tags Users, Orders
  * @param int id User ID
  * @query string status Filter by order status
  * @query int page Page number (default: 1)
  * @response 200 {"orders": [{"id": "int", "product": "string", "total": "float", "status": "string"}], "total
 // ...
```

## 13. Gotchas

### *1. Annotations Must Be Directly Above the Route*

**Fix:** Make sure the `*/` closing is on the line directly before `Router.get(...)`.

### *2. Missing @tags Makes Endpoints Hard to Find*

**Fix:** Add `@tags ResourceName` to every route doc-block.

### *3. @body Must Be Valid JSON*

**Fix:** Every key and string value must be in double quotes.

### *4. Swagger Shows Routes You Did Not Annotate*

**Fix:** By design. Add `@hidden` to hide a route from Swagger.

### *5. Response Examples Do Not Match Actual Responses*

**Fix:** Update annotations when you change handler response format.

### *6. Swagger UI Not Available in Production*

**Fix:** Set `TINA4_SWAGGER=true` explicitly in `.env`.

### *7. SDK Generation Produces Incorrect Types*

**Fix:** Use correct OpenAPI types: `"string"`, `"number"`, `"boolean"`.

# Chapter 11: Queue System

## 1. Do Not Make the User Wait

Your app sends welcome emails on signup. Generates PDF invoices. Resizes uploaded images. Each task takes 2 to 30 seconds. Run them inside the HTTP request and the user stares at a spinner.

Queues move slow work to a background process. The user gets a response in milliseconds. The work still happens -- just not during the request.

Tina4 has a built-in queue system. Works out of the box with SQLite. No Redis. No RabbitMQ. No external services.

## 2. Why Queues Matter

Without queues: 6530ms response time. With queues: 33ms. Same work done. Different timing.

Queues also deliver retry logic, rate limiting, fault isolation, and horizontal scaling.

## 3. SQLite Queue (Default)

### *Producing a Job*

```
import { Router, Queue } from "tina4-nodejs";

Router.post("/api/register", async (req, res) => {
    const body = req.body;
    const userId = 42;

    await Queue.produce("send-welcome-email", {
        user_id: userId,
        email: body.email,
        name: body.name
    });

    return res.status(201).json({
        message: "Registration successful. Welcome email will arrive shortly.",
        user_id: userId
    });
});
```

### *Producing with Options*

```
// Delay by 60 seconds
await Queue.produce("send-reminder", { user_id: 42, message: "Verify your email!" }, { delay: 60 });

// Set max retries
await Queue.produce("generate-invoice", { order_id: 101 }, { maxRetries: 5 });

// Set priority (lower = higher priority)
await Queue.produce("resize-image", { image_path: "/uploads/photo.jpg" }, { priority: 1 });
```

## 4. Consuming Jobs

```
import { Queue } from "tina4-nodejs";

Queue.consume("send-welcome-email", async (job) => {
    const { email, name } = job.payload;

    console.log(`Sending welcome email to ${email} for ${name}`);

    return true; // Mark as completed
});
```

### *Starting the Consumer*

```
tina4 queue:work
```

```
Queue worker started
  Listening on: all queues
  Backend: sqlite:///data/queue.db
  Polling interval: 1s

[2026-03-22 14:30:01] Processing job #1 on "send-welcome-email"
[2026-03-22 14:30:01] Job #1 completed in 45ms
```

### *Running Multiple Workers*

```
tina4 queue:work --queue send-welcome-email
tina4 queue:work --queue generate-invoice
```

## 5. Job Lifecycle

```
pending → reserved → completed
                   → failed → pending (retry)
                            → dead (max retries exceeded)
```

### *Queue Statistics*

```
const stats = await Queue.stats("send-welcome-email");
// { queue: "send-welcome-email", pending: 12, reserved: 2, completed: 1453, failed: 3, dead: 1 }
```

## 6. Retry Logic

Retries use exponential backoff: 10s, 30s, 90s. Default max retries: 3.

```
Queue.consume("send-welcome-email", async (job) => {
    try {
        const success = await sendEmail(job.payload.email, "Welcome!", "Welcome to our platform.");
        if (!success) {
            console.log(`Failed to send email (attempt ${job.attempts})`);
            return false; // Triggers retry
        }
        return true;
    } catch (e) {
        console.log(`Exception: ${e.message}`);
        return false;
    }
});
```

## 7. Dead Letter Queue

```
tina4 queue:dead
tina4 queue:retry 42
tina4 queue:retry --queue send-welcome-email
tina4 queue:clear --older-than 7d
```

## 8. Switching to RabbitMQ

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=5672
TINA4_QUEUE_USERNAME=guest
TINA4_QUEUE_PASSWORD=guest
```

Your code stays identical. Same `Queue.produce()` and `Queue.consume()` calls. The backend is an implementation detail.

## 9. Switching to Kafka

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=9092
TINA4_QUEUE_GROUP_ID=my-app-workers
```

## 10. Multiple Jobs from One Action

```
Router.post("/api/orders", async (req, res) => {
    const orderId = 101;

    await Queue.produce("send-order-confirmation", { order_id: orderId, email: req.body.email });
    await Queue.produce("generate-invoice", { order_id: orderId, format: "pdf" });
    await Queue.produce("update-inventory", { items: req.body.items });
    await Queue.produce("notify-warehouse", { order_id: orderId, shipping_address: req.body.shipping_address

    return res.status(201).json({ message: "Order placed successfully", order_id: orderId });
});
```

## 11. Exercise: Build an Email Queue

Create `POST /api/signup` that queues a welcome email, a consumer that logs it, and `GET /api/queue/stats`.

## 12. Solution

```
import { Router, Queue } from "tina4-nodejs";

/**
 * @noauth
 */
Router.post("/api/signup", async (req, res) => {
    const body = req.body;
    if (!body.name || !body.email || !body.password) {
```

```
        return res.status(400).json({ error: "Name, email, and password are required" });
    }

    const userId = Math.floor(Math.random() * 10000);

    await Queue.produce("welcome-emails", {
        user_id: userId,
        name: body.name,
        email: body.email,
        signed_up_at: new Date().toISOString()
    });

    return res.status(201).json({
        message: "Signup successful! A welcome email will be sent shortly.",
        user_id: userId
    });
});
// ...
```

## 13. Gotchas

### *1. Worker Must Be Running Separately*

**Fix:** Run `tina4 queue:work` in a separate terminal.

### *2. Consumer Not Registered*

**Fix:** Put `Queue.consume()` in a file inside `src/routes/`.

### *3. Job Payload Must Be JSON-Serializable*

**Fix:** Only pass simple data types. Pass IDs, not full objects.

### *4. Jobs Process in Wrong Order*

**Fix:** Use a single worker for ordered processing. Most use cases do not need ordering.

### *5. SQLite Queue Lock Contention*

**Fix:** For multiple workers, switch to RabbitMQ or Kafka.

### *6. Consumer Returns Nothing*

**Fix:** Always `return true` on success, `return false` on failure.

### *7. Dead Letter Jobs Accumulate*

**Fix:** Monitor `Queue.stats()` and set up alerts.

# Chapter 12: Real-time with WebSocket

## 1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else should see it. Now. Not after a refresh.

WebSocket establishes a persistent, bi-directional connection between the browser and the server. Data flows both ways. No polling. No refresh.

Tina4 treats WebSocket the same way it treats routing. Define a WebSocket handler the same way you define an HTTP route.

## 2. What WebSocket Is

HTTP is a conversation that ends. Request. Response. Connection closes. WebSocket is an open line. Persistent. Bi-directional. Low overhead. Real-time.

## 3. Router.websocket() -- WebSocket as a Route

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/echo", async (connection, event, data) => {
    if (event === "message") {
        connection.send(`Echo: ${data}`);
    }
});
```

```
tina4 serve

  Tina4 Node.js v3.0.0
  HTTP server running at http://0.0.0.0:7148
  WebSocket server running at ws://0.0.0.0:7148
```

## 4. Connection Events

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/chat", async (connection, event, data) => {
    switch (event) {
        case "open":
            console.log(`[Chat] New connection: ${connection.id}`);
            connection.send(JSON.stringify({
                type: "system",
                message: "Welcome to the chat!",
                your_id: connection.id
            }));
            break;

        case "message":
            const message = JSON.parse(data);
            console.log(`[Chat] ${connection.id}: ${message.text ?? data}`);
            connection.send(JSON.stringify({
                type: "message",
```

```
                from: connection.id,
                text: message.text ?? data,
                timestamp: new Date().toISOString()
            }));
            break;

        case "close":
// ...
```

## 5. Sending to a Single Client

```
connection.send(JSON.stringify({ type: "pong", timestamp: new Date().toISOString() }));
```

## 6. Broadcasting to All Clients

```
Router.websocket("/ws/announcements", async (connection, event, data) => {
    if (event === "open") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: "A new user joined",
            online_count: connection.connectionCount()
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        connection.broadcast(JSON.stringify({
            type: "announcement",
            from: connection.id,
            text: message.text ?? "",
            timestamp: new Date().toISOString()
        }));
    }

    if (event === "close") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: "A user left",
            online_count: connection.connectionCount()
        }));
// ...
```

### *Broadcast Excluding Sender*

```
connection.broadcast(JSON.stringify({ type: "message", text: message.text }), true);
```

## 7. Path-Scoped Isolation

```
Router.websocket("/ws/chat/:room", async (connection, event, data) => {
    const room = connection.params.room;

    if (event === "open") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: `Someone joined room ${room}`,
            room,
            online: connection.connectionCount()
```

```
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        connection.broadcast(JSON.stringify({
            type: "message",
            room,
            from: message.username ?? "Anonymous",
            text: message.text ?? "",
            timestamp: new Date().toISOString()
        }));
    }
});
```

Connect to different rooms:

```
ws://localhost:7148/ws/chat/general
ws://localhost:7148/ws/chat/dev-team
```

A broadcast in `/ws/chat/general` reaches only clients on that path. The rooms are walls.

## 8. Live Chat with Typing Indicators

```
import { Router } from "tina4-nodejs";

const chatUsers: Record<string, { id: string; username: string; room: string }> = {};

Router.websocket("/ws/livechat/:room", async (connection, event, data) => {
    const room = connection.params.room;

    if (event === "open") {
        chatUsers[connection.id] = { id: connection.id, username: "Anonymous", room };

        connection.send(JSON.stringify({
            type: "welcome",
            message: `Connected to room: ${room}`,
            your_id: connection.id,
            online: connection.connectionCount()
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        const type = message.type ?? "message";

        if (type === "set-username") {
            const oldName = chatUsers[connection.id].username;
            chatUsers[connection.id].username = message.username;
// ...
```

## 9. Live Notifications via HTTP

```
Router.post("/api/orders/:orderId:int/ship", async (req, res) => {
    const orderId = req.params.orderId;
    const userId = req.body.user_id ?? 0;

    Router.pushToWebSocket(`/ws/notifications/${userId}`, JSON.stringify({
        type: "notification",
```

```
        title: "Order Shipped",
        message: `Your order #${orderId} has been shipped!`,
        timestamp: new Date().toISOString()
    }));

    return res.json({ message: "Order shipped, user notified" });
});
```

## 10. Connecting from JavaScript

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);
    });

    function sendMessage(text) {
        ws.send(JSON.stringify({ type: "message", text }));
    }
</script>
```

### *Auto-Reconnect*

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,
    reconnectInterval: 3000,
    maxReconnectAttempts: 10
});
```

## 11. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat at `/ws/room/:roomName` with usernames, join/leave messages, and an HTML page at `GET /room/:roomName`.

## 12. Solution

Create `src/routes/chat-room.ts`:

```
import { Router } from "tina4-nodejs";

const roomUsers: Record<string, string> = {};

Router.websocket("/ws/room/:roomName", async (connection, event, data) => {
    const room = connection.params.roomName;
    const key = `${room}:${connection.id}`;

    if (event === "open") {
        roomUsers[key] = "Anonymous";
        connection.send(JSON.stringify({ type: "system", message: `Welcome to room: ${room}`, online: connecti
```

```
        connection.broadcast(JSON.stringify({ type: "system", message: "A new user joined", online: connectio
    }

    if (event === "message") {
        const msg = JSON.parse(data);

        if (msg.type === "set-name") {
            const oldName = roomUsers[key];
            roomUsers[key] = msg.name ?? "Anonymous";
            connection.broadcast(JSON.stringify({ type: "system", message: `${oldName} changed their name to $
        }

        if (msg.type === "chat") {
            connection.broadcast(JSON.stringify({ type: "chat", from: roomUsers[key], text: msg.text ?? "", ti
// ...
```

## 13. Gotchas

### *1. WebSocket Needs a Persistent Server*

**Fix:** Use `tina4 serve` or `npx tsx app.ts`. Configure Nginx for WebSocket proxying.

### *2. Messages Are Strings, Not Objects*

**Fix:** Always `JSON.parse(data)` on receive, `JSON.stringify()` on send.

### *3. Connection Count Is Per-Path*

**Fix:** By design. Each path is an isolated group.

### *4. Broadcasting Does Not Scale Across Servers*

**Fix:** Use Redis pub/sub to relay messages across server instances.

### *5. Large Messages Cause Disconnects*

**Fix:** Keep messages under 64KB. Use HTTP for bulk data.

### *6. Memory Leak from Tracking Users*

**Fix:** Always clean up in the `close` handler.

### *7. No Authentication on WebSocket*

**Fix:** Pass token as query parameter and validate in the `open` handler.

# Chapter 13: Email with Messenger

## 1. Every App Sends Email

Signup confirmations. Password resets. Weekly digests. Every application sends email.

Tina4's `Messenger` class handles SMTP configuration, HTML templates, attachments, and delivery. In development mode, Tina4 intercepts all outgoing emails and shows them in the dev dashboard. Nothing leaves the server until you say so.

## 2. Messenger Configuration via .env

```
TINA4_MAIL_SMTP_HOST=smtp.example.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@example.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
TINA4_MAIL_FROM_ADDRESS=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

### *Common Provider Configurations*

**Gmail:**

```
TINA4_MAIL_SMTP_HOST=smtp.gmail.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

**SendGrid:**

```
TINA4_MAIL_SMTP_HOST=smtp.sendgrid.net
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=apikey
TINA4_MAIL_SMTP_PASSWORD=your-sendgrid-api-key
```

## 3. Constructor Override Pattern

```
import { Messenger } from "tina4-nodejs";

// Uses .env defaults
const mailer = new Messenger();

// Override specific settings
const marketingMailer = new Messenger({
    host: "smtp.mailgun.org",
    port: 587,
    username: "marketing@mg.yourdomain.com",
    password: "marketing-smtp-password",
    encryption: "tls",
    fromAddress: "newsletter@yourdomain.com",
    fromName: "My Store Newsletter"
});
```

## 4. Sending Plain Text Email

```
import { Router, Messenger } from "tina4-nodejs";

Router.post("/api/contact", async (req, res) => {
    const body = req.body;
    const mailer = new Messenger();

    const result = await mailer.send(
        body.email,
        "Contact Form Submission",
        `Name: ${body.name}\nEmail: ${body.email}\nMessage:\n${body.message}`
    );

    if (result.success) {
        return res.json({ message: "Email sent successfully" });
    }
    return res.status(500).json({ error: "Failed to send email", details: result.error });
});
```

## 5. Sending HTML Email with Text Fallback

```
const result = await mailer.send(
    "alice@example.com",
    "Welcome to My Store!",
    htmlBody,
    { textBody: textBody }
);
```

## 6. Adding Attachments

```
const result = await mailer.send(
    "accounting@example.com",
    "Monthly Invoice #1042",
    "<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    {
        attachments: [
            "/path/to/invoices/invoice-1042.pdf",
            { path: "/tmp/export.csv", name: "my-store-export.csv" }
        ]
    }
);
```

## 7. CC and BCC

```
const result = await mailer.send(
    "alice@example.com",
    "Team Meeting Notes",
    "<p>Here are the notes.</p>",
    {
        cc: ["bob@example.com", "charlie@example.com"],
        bcc: ["manager@example.com"],
        replyTo: "alice@example.com"
    }
);
```

## 8. Reading Inbox via IMAP

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
TINA4_MAIL_IMAP_USERNAME=support@example.com
TINA4_MAIL_IMAP_PASSWORD=your-imap-password
TINA4_MAIL_IMAP_ENCRYPTION=ssl
```

```
Router.get("/api/inbox", async (req, res) => {
    const mailer = new Messenger();
    const emails = await mailer.getInbox({ limit: 20, unreadOnly: true });

    const messages = emails.map(email => ({
        id: email.id,
        from: email.from,
        subject: email.subject,
        date: email.date,
        preview: email.textBody.substring(0, 200),
        has_attachments: email.attachments.length > 0
    }));

    return res.json({ messages, count: messages.length });
});
```

## 9. Dev Mode: Email Interception

When `TINA4_DEBUG=true`, Tina4 catches all outgoing emails and holds them in the dev dashboard. Navigate to `/tina4/console` to inspect them. No email reaches a real inbox during development.

Override with `TINA4_MAIL_INTERCEPT=false` if you need real delivery in debug mode.

## 10. Using Templates for Email Content

Create `src/templates/emails/welcome.html` with Frond template syntax, then render and send:

```
import { Router, Messenger, Frond } from "tina4-nodejs";

Router.post("/api/register", async (req, res) => {
    const body = req.body;
    const userId = 42;

    const htmlBody = await Frond.render("emails/welcome.html", {
        name: body.name,
        email: body.email,
        user_id: userId,
        base_url: process.env.APP_URL ?? "http://localhost:7148",
        app_name: "My Store",
        promo_code: "WELCOME10"
    });

    const mailer = new Messenger();
    const result = await mailer.send(body.email, `Welcome to My Store, ${body.name}!`, htmlBody);

    return res.status(201).json({ message: "Registration successful", email_sent: result.success, user_id: use
});
```

## 11. Sending Email via Queues

```
import { Router, Queue, Messenger, Frond } from "tina4-nodejs";

Router.post("/api/register", async (req, res) => {
    const userId = 42;
    await Queue.produce("emails", {
        template: "emails/welcome.html",
        to: req.body.email,
        subject: `Welcome to My Store, ${req.body.name}!`,
        data: { name: req.body.name, email: req.body.email, user_id: userId, app_name: "My Store" }
    });
    return res.status(201).json({ message: "Registration successful", user_id: userId });
});

Queue.consume("emails", async (job) => {
    const { template, to, subject, data } = job.payload;
    const htmlBody = await Frond.render(template, data);
    const mailer = new Messenger();
    const result = await mailer.send(to, subject, htmlBody);
    if (!result.success) {
        console.log(`Email failed: ${result.error}`);
        return false;
    }
    return true;
});
```

## 12. Exercise: Build a Contact Form with Email Notification

Create `GET /contact` page, `POST /contact` endpoint that validates, sends email, and shows a flash message.

## 13. Solution

Create `src/routes/contact.ts`:

```
import { Router, Messenger, Frond } from "tina4-nodejs";

Router.get("/contact", async (req, res) => {
    const flash = req.session._flash ?? null;
    delete req.session._flash;
    return res.html("contact.html", { flash });
});

Router.post("/contact", async (req, res) => {
    const body = req.body;
    const errors: string[] = [];
    if (!body.name) errors.push("Name is required");
    if (!body.email) errors.push("Email is required");
    if (!body.subject) errors.push("Subject is required");
    if (!body.message) errors.push("Message is required");

    if (errors.length > 0) {
        req.session._flash = { type: "error", message: `Please fill in all fields: ${errors.join(", ")}` };
        return res.redirect("/contact");
    }

    const htmlBody = await Frond.render("emails/contact-notification.html", {
```

```
        name: body.name, email: body.email, subject: body.subject,
        message: body.message, submitted_at: new Date().toLocaleString()
    });
// ...
```

## 14. Gotchas

### *1. Gmail Blocks "Less Secure" Apps*

**Fix:** Generate an "App Password" in Google Account settings.

### *2. Emails Go to Spam*

**Fix:** Use proper SPF, DKIM, DMARC records. Use a transactional email service.

### *3. HTML Email Looks Broken*

**Fix:** Use inline styles. Use table-based layouts. Test across clients.

### *4. Attachment File Not Found*

**Fix:** Use absolute paths. Verify the file exists before sending.

### *5. Dev Mode Silently Intercepts Emails*

**Fix:** Check the dev dashboard at `/tina4/console`.

### *6. Template Variables Not Substituted*

**Fix:** Use `Frond.render()`, not `fs.readFileSync()`.

### *7. Connection Timeout on Send*

**Fix:** Test SMTP connectivity. Verify hostname, port, and encryption.

# Chapter 14: Caching

## 1. From 800ms to 3ms

Your product catalog page runs 12 database queries. 800 milliseconds to render. Every visitor triggers the same queries for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each.

Tina4 caches at three levels: response caching, database query caching, and a direct cache API.

## 2. Response Caching with ResponseCache Middleware

```
import { Router } from "tina4-nodejs";

Router.get("/api/products", async (req, res) => {
    console.log("Handler called -- should only appear once every 5 minutes");

    const products = [
        { id: 1, name: "Wireless Keyboard", price: 79.99 },
        { id: 2, name: "USB-C Hub", price: 49.99 }
    ];

    return res.json({ products, generated_at: new Date().toISOString() });
}, "ResponseCache:300");
```

The `"ResponseCache:300"` caches the response for 300 seconds.

### *Cache Headers*

```
X-Cache: HIT
X-Cache-TTL: 247
Cache-Control: public, max-age=300
```

### *What Not to Cache*

Do not use `ResponseCache` on POST/PUT/DELETE routes, user-specific endpoints, or real-time data.

## 3. Memory Cache (Default)

```
TINA4_CACHE_BACKEND=memory
```

The fastest option. Data lives in memory. Server restart wipes it clean.

## 4. Redis Cache

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_HOST=localhost
TINA4_CACHE_PORT=6379
TINA4_CACHE_PREFIX=myapp:cache:
```

## 5. File Cache

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_PATH=/path/to/cache/directory
```

## 6. Direct Cache API

```
import { cacheGet, cacheSet, cacheDelete } from "tina4-nodejs";

// Set a value with TTL
await cacheSet("product:42", { id: 42, name: "Keyboard", price: 79.99 }, 300);

// Get a value
const product = await cacheGet("product:42");

// Delete a value
await cacheDelete("product:42");
```

### *Cache-Aside Pattern*

```
import { Router, Database, cacheGet, cacheSet } from "tina4-nodejs";

Router.get("/api/products/:id:int", async (req, res) => {
    const id = req.params.id;
    const cacheKey = `product:${id}`;

    let product = await cacheGet(cacheKey);

    if (product !== null) {
        return res.json({ ...product, source: "cache" });
    }

    const db = Database.getConnection();
    product = await db.fetchOne("SELECT * FROM products WHERE id = :id", { id });

    if (product === null) {
        return res.status(404).json({ error: "Product not found" });
    }

    await cacheSet(cacheKey, product, 600);

    return res.json({ ...product, source: "database" });
});
```

## 7. Database Query Caching

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=300
```

Identical queries with identical parameters return cached results. The cache key is the SQL statement plus its parameters. Change either and the cache treats it as a new query.

## 8. Cache Invalidation Strategies

### *Time-Based Expiry (TTL)*

```
await cacheSet("products:featured", featuredProducts, 600);
```

### *Event-Based Invalidation*

```
Router.put("/api/products/:id:int", async (req, res) => {
    // Update database...
    await cacheDelete(`product:${req.params.id}`);
    await cacheDelete("products:all");
    // ...
});
```

### *Write-Through Cache*

```
Router.put("/api/products/:id:int", async (req, res) => {
    // Update database...
    const updated = await db.fetchOne("SELECT * FROM products WHERE id = :id", { id });
    await cacheSet(`product:${id}`, updated, 600);
    return res.json(updated);
});
```

## 9. TTL Management

| Data Type | Suggested TTL |

|----------|----------|

| Static config | 3600 (1 hour) |

| Product catalog | 300 (5 min) |

| User profile | 60 (1 min) |

| Search results | 120 (2 min) |

| Shopping cart | 0 (no cache) |

## 10. Cache Statistics

```
import { cacheStats } from "tina4-nodejs";

Router.get("/api/cache/stats", async (req, res) => {
    return res.json(await cacheStats());
});
```

## 11. Exercise: Cache an Expensive Product Listing

Build `GET /api/store/products` with direct cache API (5-min TTL), `POST /api/store/products` that invalidates cache, and `GET /api/store/cache-stats`.

## 12. Solution

```
import { Router, cacheGet, cacheSet, cacheDelete, cacheStats } from "tina4-nodejs";
import { createHash } from "crypto";

function getProductStore() {
    return [
        { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
```

```
        { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
        { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false }
    ];
}

Router.get("/api/store/products", async (req, res) => {
    const category = req.query.category ?? null;
    const page = parseInt(req.query.page ?? "1", 10);
    const limit = parseInt(req.query.limit ?? "20", 10);

    const cacheKey = `store:products:${createHash("md5").update(JSON.stringify({ category, page, limit })).di

    const cached = await cacheGet(cacheKey);
    if (cached !== null) {
        return res.json({ ...cached, source: "cache" });
    }

    let products = getProductStore();
    if (category) {
// ...
```

## 13. Gotchas

### *1. Caching Authenticated Responses*

**Fix:** Do not use `ResponseCache` on user-specific endpoints. Use user-specific cache keys.

### *2. Cache Stampede*

**Fix:** Tina4's `ResponseCache` handles this with stale-while-revalidate.

### *3. Memory Cache Lost on Restart*

**Fix:** Use Redis for production.

### *4. Stale Data After Database Update*

**Fix:** Always invalidate or update cache after writes.

### *5. Cache Key Collisions*

**Fix:** Include all relevant parameters in the cache key.

### *6. Serialization Overhead*

**Fix:** Only cache data that is expensive to compute.

### *7. Forgetting to Set TTL*

**Fix:** Always set a TTL to prevent unbounded memory growth.

# Chapter 15: Frontend Integration

## 1. Beyond JSON APIs

Your API returns perfect JSON. Now someone has to build the interface.

Tina4 ships two frontend tools: **tina4css** (a utility CSS framework) and **frond.js** (a reactive JavaScript library). Both arrive with every project. No npm installs. No build tools. No webpack.

## 2. tina4css -- The Built-In CSS Framework

`tina4.css` lives at `/css/tina4.css`. It delivers layout, typography, buttons, cards, forms, alerts, and utility classes.

### *Layout*

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
</div>
```

### *Components*

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-danger">Danger</button>

<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content</div>
</div>

<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>

<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

### *Utility Classes*

```
<p class="text-center">Centered</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding</div>
<span class="text-muted">Gray text</span>
```

## 3. frond.js -- Reactive JavaScript

`frond.js` lives at `/js/frond.js`. It delivers reactive data binding, DOM manipulation, HTTP fetch helpers, and WebSocket support.

### *Reactive Data Binding*

```
<div id="app">
```

```
    <h1>{{ title }}</h1>
    <p>Count: {{ count }}</p>
    <button onclick="increment()">+1</button>
</div>

<script src="/js/frond.js"></script>
<script>
    const app = frond.reactive({
        el: "#app",
        data: {
            title: "My Counter",
            count: 0
        }
    });

    function increment() {
        app.data.count++;
    }
</script>
```

Changes to `app.data.count` automatically update the DOM.

### *HTTP Fetch Helper*

```
// GET request
const products = await frond.get("/api/products");

// POST request
const result = await frond.post("/api/products", {
    name: "Widget",
    price: 9.99
});

// PUT request
await frond.put("/api/products/1", { price: 12.99 });

// DELETE request
await frond.del("/api/products/1");
```

### *WebSocket Helper*

```
const ws = frond.ws("/ws/chat/general");

ws.on("open", () => console.log("Connected"));
ws.on("message", (data) => console.log(JSON.parse(data)));
ws.on("close", () => console.log("Disconnected"));

ws.send(JSON.stringify({ type: "message", text: "Hello!" }));
```

## 4. Building a CRUD Interface

Create `src/templates/product-manager.html`:

```
{% extends "base.html" %}

{% block title %}Product Manager{% endblock %}

{% block content %}
    <h1>Product Manager</h1>

    <div id="app">
        <form id="add-form" class="card" style="padding: 16px; margin-bottom: 16px;">
```

```
            <h3>Add Product</h3>
            <div class="form-group">
                <label for="name">Name</label>
                <input type="text" id="name" class="form-control" required>
            </div>
            <div class="form-group">
                <label for="price">Price</label>
                <input type="number" id="price" class="form-control" step="0.01" required>
            </div>
            <button type="submit" class="btn btn-primary">Add Product</button>
        </form>

        <div id="product-list"></div>
    </div>

    <script src="/js/frond.js"></script>
// ...
```

## 5. SCSS Compilation

Place `.scss` files in `src/public/scss/`. Tina4 compiles them automatically:

```
src/public/scss/custom.scss → /css/custom.css
```

## 6. Static File Serving

Files in `src/public/` are served directly:

```
src/public/images/logo.png → /images/logo.png
src/public/js/app.js       → /js/app.js
src/public/css/custom.css  → /css/custom.css
```

## 7. Integrating with React, Vue, or Svelte

Tina4 is the API backend. Point your frontend build tool's output to `src/public/`:

```
# Vue
VITE_OUTPUT_DIR=../my-tina4-project/src/public

# React (Create React App)
BUILD_PATH=../my-tina4-project/src/public
```

Or use CORS to run them on separate ports during development:

```
CORS_ORIGINS=http://localhost:3000,http://localhost:5173
```

## 8. Exercise: Build a Product Dashboard

Build a single-page product dashboard using frond.js that displays products in a grid, supports adding and deleting products, and shows real-time updates via WebSocket.

## 9. Solution

Create `src/routes/dashboard-page.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/dashboard", async (req, res) => {
    return res.html("dashboard.html", {});
});
```

Create `src/templates/dashboard.html` with a full frond.js-powered dashboard using reactive data binding, HTTP helpers for CRUD operations, and WebSocket for live updates.

## 10. Gotchas

### *1. CORS Errors with Separate Frontend*

**Fix:** Set `CORS_ORIGINS` in `.env`.

### *2. frond.js Not Loading*

**Fix:** Ensure `<script src="/js/frond.js"></script>` is in your template. The file is auto-provided.

### *3. Static Files Return 404*

**Fix:** Files must be in `src/public/`, not `public/` at the project root.

### *4. SCSS Not Compiling*

**Fix:** Place files in `src/public/scss/` and restart the server.

### *5. Cache Busting*

**Fix:** Use versioned filenames or query strings: `/css/app.css?v=1.2.3`.

### *6. React Router Conflicts*

**Fix:** Add a catch-all route that serves `index.html` for client-side routing.

### *7. Large Bundle Sizes*

**Fix:** Use code splitting. Serve static assets from a CDN.

# Chapter 16: GraphQL

## 1. The Problem GraphQL Solves

Your mobile app needs a list of products with name and price. 2KB. Your REST API returns all 20 fields. 50KB. Twenty-five times the payload the client needs.

GraphQL flips control. The client asks for the fields it wants. The server returns nothing else.

Tina4 includes a built-in GraphQL engine. Zero external packages.

## 2. GraphQL vs REST

| Aspect | REST | GraphQL |

|--------|------|---------|

| Endpoints | One per resource | One endpoint (`/graphql`) |

| Data shape | Server decides | Client decides |

| Over-fetching | Common | Never |

| Under-fetching | Common | Never |

## 3. Enabling GraphQL

GraphQL runs at `/graphql` by default. Test:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

## 4. Defining a Schema

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

## 5. Writing Resolvers

Create `src/graphql/resolvers.ts`:

```
import { GraphQL } from "tina4-nodejs";
import { Product } from "../orm/Product";

GraphQL.resolve("Query", "products", async (root, args) => {
    const product = new Product();
    const products = await product.select("*", "", {}, "name ASC");
    return products.map(p => p.toDict());
});

GraphQL.resolve("Query", "product", async (root, args) => {
    const product = new Product();
    await product.load(args.id);
    return product.id ? product.toDict() : null;
});
```

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true}
    ]
  }
}
```

## 6. Mutations

```
input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

type Mutation {
    createProduct(input: ProductInput!): Product!
    deleteProduct(id: Int!): DeleteResult!
}

type DeleteResult {
    success: Boolean!
    message: String!
}
```

```
GraphQL.resolve("Mutation", "createProduct", async (root, args) => {
    const input = args.input;
    const product = new Product();
    product.name = input.name;
    product.category = input.category ?? "Uncategorized";
    product.price = parseFloat(input.price);
    product.inStock = Boolean(input.inStock ?? true);
    await product.save();
    return product.toDict();
});

GraphQL.resolve("Mutation", "deleteProduct", async (root, args) => {
    const product = new Product();
    await product.load(args.id);
```

```
    if (!product.id) return { success: false, message: "Product not found" };
    await product.delete();
    return { success: true, message: "Product deleted" };
});
```

## 7. Nested Types and Relationships

```
GraphQL.resolve("Post", "author", async (post, args) => {
    const user = new User();
    await user.load(post.user_id);
    return user.toDict();
});

GraphQL.resolve("Post", "comments", async (post, args) => {
    const comment = new Comment();
    const comments = await comment.select("*", "post_id = :postId", { postId: post.id });
    return comments.map(c => c.toDict());
});

GraphQL.resolve("User", "posts", async (user, args) => {
    const post = new Post();
    const posts = await post.select("*", "user_id = :userId", { userId: user.id });
    return posts.map(p => p.toDict());
});
```

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name email } comments { authorName body } } }"}'
```

## 8. Auto-Generating Schema from ORM Models

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

Every ORM model with `static autoCrud = true` gets GraphQL types and resolvers generated for it. Zero manual schema writing.

## 9. The GraphiQL Playground

When `TINA4_DEBUG=true`:

```
http://localhost:7148/graphql/playground
```

## 10. Query Variables

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

## 11. Exercise: Build a GraphQL API for a Blog

Define types for User, Post, Comment. Implement queries (posts, post, user) and mutations (createPost, addComment).

## 12. Solution

Create `src/graphql/blog-schema.graphql` with User, Post, Comment types, and resolvers in `src/graphql/blog-resolvers.ts` using the ORM models from Chapter 6.

```
import { GraphQL } from "tina4-nodejs";
import { Post } from "../orm/Post";
import { User } from "../orm/User";
import { Comment } from "../orm/Comment";

GraphQL.resolve("Query", "posts", async () => {
    const post = new Post();
    const posts = await post.select("*", "published = :pub", { pub: 1 }, "created_at DESC");
    return posts.map(p => p.toDict());
});

GraphQL.resolve("Query", "post", async (root, args) => {
    const post = new Post();
    await post.load(args.id);
    return post.id ? post.toDict() : null;
});

GraphQL.resolve("Query", "user", async (root, args) => {
    const user = new User();
    await user.load(args.id);
    return user.id ? user.toDict() : null;
});

GraphQL.resolve("Post", "author", async (post) => {
    const user = new User();
// ...
```

## 13. Gotchas

***1. Schema File Not Found -- Place `.graphql` files in `src/graphql/`.***

***2. Resolver Not Called -- Check type/field names match schema exactly (case-sensitive).***

***3. Nested Resolver Returns Wrong Data -- Load the related record using the foreign key from the parent.***

***4. Mutation Input Not Parsed -- Match the query structure to the schema signature.***

***5. N+1 Query Problem -- Use data loaders or pre-load in parent resolvers.***

***6. GraphQL Playground Returns 404 -- Set `TINA4_DEBUG=true`.***

***7. Type Mismatch -- Cast values explicitly: `parseInt()`, `parseFloat()`, `Boolean()`.***

# Chapter 17: Testing

## 1. Why Tests Matter More Than You Think

Friday afternoon. Your client reported a critical bug. You fix it -- one line. But did that fix break something else? The test suite answers in 2 seconds.

```
npm test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product

  AuthTest
    [PASS] test_login_with_valid_credentials
    [PASS] test_login_with_invalid_password
    [PASS] test_protected_route_without_token

  7 tests, 7 passed, 0 failed (0.34s)
```

Tina4 ships an inline testing framework. No external packages. No Jest configuration. No setup ceremony.

## 2. Your First Test

Tests live in the `tests/` directory. The test runner is `tests/run-all.ts`.

Create `tests/BasicTest.ts`:

```
import { Test } from "tina4-nodejs";

export class BasicTest extends Test {
    async testAddition() {
        this.assertEqual(2 + 2, 4, "Basic addition should work");
    }

    async testStringConcatenation() {
        const result = "Hello" + " " + "World";
        this.assertEqual(result, "Hello World", "String concatenation should work");
    }

    async testArrayLength() {
        const items = [1, 2, 3];
        this.assertEqual(items.length, 3, "Array should have 3 items");
    }

    async testBooleanLogic() {
        this.assertTrue(true, "true should be true");
        this.assertFalse(false, "false should be false");
    }
}
```

Run it:

```
npm test

Running tests...
```

```
  BasicTest
    [PASS] testAddition
    [PASS] testStringConcatenation
    [PASS] testArrayLength
    [PASS] testBooleanLogic

  4 tests, 4 passed, 0 failed (0.01s)
```

## 3. Assertion Methods

| Method | Description |

|--------|-------------|

| `assertEqual(actual, expected, message)` | Values are equal |

| `assertNotEqual(actual, expected, message)` | Values are not equal |

| `assertTrue(value, message)` | Value is truthy |

| `assertFalse(value, message)` | Value is falsy |

| `assertNull(value, message)` | Value is null |

| `assertNotNull(value, message)` | Value is not null |

| `assertContains(haystack, needle, message)` | String or array contains value |

| `assertThrows(fn, message)` | Function throws an error |

## 4. Testing Routes with testGet() and testPost()

```
import { Test } from "tina4-nodejs";

export class ProductApiTest extends Test {
    async testListProducts() {
        const response = await this.testGet("/api/products");

        this.assertEqual(response.status, 200, "Should return 200");
        this.assertNotNull(response.body.products, "Should have products array");
    }

    async testCreateProduct() {
        const response = await this.testPost("/api/products", {
            name: "Test Widget",
            category: "Testing",
            price: 9.99
        });

        this.assertEqual(response.status, 201, "Should return 201");
        this.assertEqual(response.body.name, "Test Widget", "Name should match");
        this.assertEqual(response.body.price, 9.99, "Price should match");
    }

    async testGetProductNotFound() {
        const response = await this.testGet("/api/products/99999");

// ...
```

## 5. Testing with Authentication

```
import { Test, Auth } from "tina4-nodejs";

export class AuthTest extends Test {
    private token: string = "";

    async setup() {
        // Register and login to get a token
        await this.testPost("/api/register", {
            name: "Test User",
            email: "test@example.com",
            password: "securePass123"
        });

        const loginResponse = await this.testPost("/api/login", {
            email: "test@example.com",
            password: "securePass123"
        });

        this.token = loginResponse.body.token;
    }

    async testProfileWithToken() {
        const response = await this.testGet("/api/profile", {
            headers: { Authorization: `Bearer ${this.token}` }
        });
// ...
```

## 6. Testing ORM Models

```
import { Test } from "tina4-nodejs";
import { Product } from "../src/orm/Product";

export class ProductModelTest extends Test {
    async testCreateAndLoad() {
        const product = new Product();
        product.name = "Test Product";
        product.price = 29.99;
        product.category = "Testing";
        await product.save();

        this.assertNotNull(product.id, "Should have an ID after save");

        const loaded = new Product();
        await loaded.load(product.id);

        this.assertEqual(loaded.name, "Test Product", "Name should match");
        this.assertEqual(loaded.price, 29.99, "Price should match");

        await product.delete();
    }

    async testSoftDelete() {
        const product = new Product();
        product.name = "Deletable";
// ...
```

## 7. Test Database Isolation

Tina4 creates a test database (`data/test.db`) when running tests. Each test class gets a clean database. No leftover data from previous runs.

```
# In .env.test (optional)
DATABASE_URL=sqlite:///data/test.db
```

## 8. Setup and Teardown

```
export class MyTest extends Test {
    async setup() {
        // Runs before each test
    }

    async teardown() {
        // Runs after each test
    }

    async setupOnce() {
        // Runs once before all tests in this class
    }

    async teardownOnce() {
        // Runs once after all tests in this class
    }
}
```

## 9. Running Specific Tests

```
npm test -- --filter ProductTest
npm test -- --filter testCreateProduct
```

## 10. Exercise: Write Tests for a Notes API

Write a test class that covers: creating a note, listing notes, getting a single note, updating a note, deleting a note, and validation errors.

## 11. Solution

```
import { Test } from "tina4-nodejs";

export class NotesApiTest extends Test {
    private noteId: number = 0;

    async testCreateNote() {
        const response = await this.testPost("/api/notes", {
            title: "Test Note",
            content: "This is a test note",
            tag: "testing"
        });
        this.assertEqual(response.status, 201);
        this.assertEqual(response.body.title, "Test Note");
```

```
        this.noteId = response.body.id;
    }

    async testListNotes() {
        const response = await this.testGet("/api/notes");
        this.assertEqual(response.status, 200);
        this.assertTrue(response.body.count >= 1, "Should have at least one note");
    }

    async testGetNote() {
        const response = await this.testGet(`/api/notes/${this.noteId}`);
        this.assertEqual(response.status, 200);
// ...
```

## 12. Gotchas

***1. Tests Must Extend Test Class -- `export class MyTest extends Test`.***

***2. Test Methods Must Start with "test" -- `testCreateProduct`, not `createProductTest`.***

***3. All Tests Are Async -- Use `async` and `await` for all test methods.***

***4. Test Order Is Not Guaranteed -- Do not depend on test execution order.***

***5. Database State Leaks Between Tests -- Use `setup()` and `teardown()` to clean up.***

***6. Missing Assertions -- A test without assertions passes silently. Always assert.***

***7. Test Files Must Export Classes -- `export class MyTest extends Test`, not a default export.***

# Chapter 18: Dev Tools

## 1. Debugging at 2am

2am. Production monitoring pings you -- a 500 error on checkout. You pull up the dev dashboard. Find the failing request. Full stack trace with source context. Line 47 of `src/routes/checkout.ts` -- a null reference. Add a null check. Push the fix. Back to sleep. Total time: 30 seconds.

Tina4's dev tools are part of the framework from day one. When `TINA4_DEBUG=true`, you get a development dashboard, error overlay, live reload, request inspector, and SQL query runner.

## 2. Enabling the Dev Dashboard

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Navigate to:

```
http://localhost:7148/tina4/console
```

## 3. Dashboard Overview

### *System Overview*

• Framework version, Node.js version, uptime, memory usage

• Database status, connection info

• Environment variables (sensitive values masked)

### *Request Inspector*

• Recent HTTP requests with method, path, status, duration

• Click any request to see headers, body, database queries, template renders

### *Error Log*

• Unhandled exceptions with stack traces

• Occurrence counts and timestamps

### *Queue Manager*

• Queue status: pending, reserved, completed, failed, dead counts

• Recent jobs with status and duration

### *WebSocket Monitor*

• Active WebSocket connections with metadata

• Message history

### *Routes*

• All registered routes with methods, paths, middleware, auth status

### *Mail*

• Intercepted emails with To, Subject, HTML body, attachments

## 4. Debug Overlay

When `TINA4_DEBUG=true`, HTML pages show a toolbar at the bottom:

• Request details (method, URL, duration)

• Database queries (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

## 5. Live Reload

The dev server watches `src/` for file changes and automatically reloads:

```
tina4 serve
```

When you save a `.ts` file in `src/routes/`, the server restarts and your changes are live immediately.

## 6. SQL Query Runner

The dev dashboard includes a SQL query runner. Type any SQL query. Execute it against your database. Results appear in the browser:

```
SELECT * FROM products WHERE price > 50 ORDER BY name;
```

Results render in a table. Faster than opening a separate database client for quick queries.

## 7. Error Pages

In debug mode, errors show a detailed page with:

• The exception message

• The full stack trace with source code context (5 lines above and below)

• Request details (method, URL, headers, body)

• Environment info

In production (`TINA4_DEBUG=false`), errors show a generic "Internal Server Error" page. Custom error pages go in `src/templates/errors/`:

```
src/templates/errors/404.html
src/templates/errors/500.html
```

## 8. Logging

```
import { Logger } from "tina4-nodejs";

Logger.debug("Debug message");
Logger.info("Info message");
Logger.warning("Warning message");
Logger.error("Error message");
```

Log levels are controlled by `TINA4_LOG_LEVEL` in `.env`:

| Level | Shows |

|-------|-------|

| `ALL` | Everything |

| `DEBUG` | Debug and above |

| `INFO` | Info and above |

| `WARNING` | Warning and above |

| `ERROR` | Errors only |

| `NONE` | Nothing |

Logs are written to `logs/app.log` and to stdout.

## 9. Health Check

```
http://localhost:7148/health
```

Returns system status: database connectivity, uptime, and version. Your monitoring tools and load balancers hit this endpoint.

## 10. Exercise: Explore the Dev Dashboard

1. Enable the dev dashboard

2. Make several requests to different endpoints

3. Inspect a request in the Request Inspector

4. Run a SQL query in the Query Runner

5. Trigger a 404 and inspect the error log

6. Check the queue statistics

## 11. Gotchas

### *1. Dashboard Not Available -- Check `TINA4_CONSOLE=true` and `TINA4_CONSOLE_TOKEN`.*

*2. Debug Overlay Shows in Production -- Set `TINA4_DEBUG=false`.*

*3. Console Token in Version Control -- Add `.env` to `.gitignore`.*

*4. Live Reload Not Working -- Ensure `tina4 serve` is running (not `npx tsx app.ts`).*

*5. Logs Fill Up Disk -- Set `TINA4_LOG_LEVEL=WARNING` in production.*

*6. SQL Runner in Production -- Never enable the console in production.*

*7. Performance Overhead -- Debug mode adds overhead. Always disable in production.*

# Chapter 19: CLI & Scaffolding

## 1. Getting a New Developer Up to Speed

A new developer joins your team. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line.

The Tina4 CLI is a single Rust binary. It manages all four Tina4 frameworks: PHP, Python, Ruby, and Node.js. Same commands across all languages.

## 2. tina4 init -- Project Scaffolding

```
tina4 init my-project
```

```
Creating Tina4 project in ./my-project ...
  Detected language: Node.js (package.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/public/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
  npm install
  tina4 serve
```

### *Language Detection*

The CLI detects the language from the project directory:

| File Found | Language |

|----------|----------|

| `package.json` | Node.js |

| `composer.json` | PHP |

| `pyproject.toml` | Python |

| `Gemfile` | Ruby |

## 3. tina4 serve -- Dev Server

```
tina4 serve
```

```
  Tina4 Node.js v3.0.0
  Server running at http://0.0.0.0:7148
```

```
  Debug mode: ON
  Live reload: ON
```

Options:

```
tina4 serve --port 8080
tina4 serve --host 127.0.0.1
tina4 serve --no-reload
```

# 4. tina4 generate -- Code Generation

### *Generate a Model*

```
tina4 generate model Product
```

Creates `src/orm/Product.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";

    id!: number;
    createdAt!: string;
    updatedAt!: string;
}
```

### *Generate a Route*

```
tina4 generate route products
```

Creates `src/routes/products.ts` with GET, POST, PUT, DELETE stubs.

### *Generate a Migration*

```
tina4 generate migration create_products_table
```

Creates `src/migrations/20260322143000_create_products_table.sql`.

### *Generate a Test*

```
tina4 generate test ProductTest
```

Creates `tests/ProductTest.ts` with test stubs.

# 5. tina4 migrate -- Database Migrations

```
tina4 migrate              # Apply pending migrations
tina4 migrate:status       # Show migration status
tina4 migrate:rollback     # Roll back last migration
tina4 migrate:create NAME  # Create a new migration
```

# 6. tina4 routes -- Route Listing

```
tina4 routes
tina4 routes --method POST
tina4 routes --filter users
```

## 7. tina4 queue -- Queue Management

```
tina4 queue:work                        # Start processing jobs
tina4 queue:work --queue send-email     # Process specific queue
tina4 queue:dead                        # List dead letter jobs
tina4 queue:retry 42                    # Retry a dead job
tina4 queue:clear --older-than 7d       # Clear old dead jobs
```

## 8. tina4 test -- Running Tests

```
tina4 test                      # Run all tests
tina4 test --filter ProductTest # Run specific test class
```

Or use npm:

```
npm test
```

This runs `tests/run-all.ts` which discovers and executes all test classes.

## 9. tina4 doctor -- System Check

```
tina4 doctor
```

```
System Check
  Node.js: v20.11.1 [OK]
  npm: 10.2.4 [OK]
  tsx: 4.7.0 [OK]
  SQLite: 3.43.0 [OK]
  .env: found [OK]
  Database: connected [OK]
  AI tools detected:
    Claude Code: .claude/ [FOUND]
    Cursor: .cursor/ [NOT FOUND]
```

## 10. tina4 build -- Production Build

```
tina4 build
```

Compiles TypeScript to JavaScript. Bundles for production. Optimizes:

```
Building for production...
  Compiled 47 TypeScript files
  Output: dist/
  Size: 245KB

Ready for deployment:
  node dist/app.js
```

## 11. Custom CLI Commands

Create custom commands by adding files to `src/commands/`:

```
// src/commands/seed-products.ts
```

```
import { Command, Database } from "tina4-nodejs";

export default class SeedProducts extends Command {
    static command = "seed:products";
    static description = "Seed the database with sample products";

    async run() {
        const db = Database.getConnection();

        const products = [
            { name: "Keyboard", price: 79.99 },
            { name: "Mouse", price: 29.99 },
            { name: "Monitor", price: 399.99 }
        ];

        for (const p of products) {
            await db.execute(
                "INSERT INTO products (name, price) VALUES (:name, :price)",
                p
            );
        }

        console.log(`Seeded ${products.length} products`);
    }
// ...
```

Run it:

```
tina4 seed:products
```

## 12. Exercise: Scaffold a Blog Project

Use the CLI to scaffold a complete blog project from scratch.

### *Requirements*

1. Initialize a new project called `my-blog`
2. Generate models: User, Post, Comment
3. Generate migrations for each model
4. Generate routes for posts and comments
5. Run migrations
6. Generate tests for the Post API
7. Run the test suite

## 13. Solution

```
tina4 init my-blog
cd my-blog
npm install

tina4 generate model User
tina4 generate model Post
tina4 generate model Comment
```

```
tina4 generate migration create_users_table
tina4 generate migration create_posts_table
tina4 generate migration create_comments_table

# Edit migrations to add columns, then:
tina4 migrate

tina4 generate route posts
tina4 generate route comments

tina4 generate test PostApiTest

npm test

tina4 serve
```

## 14. Gotchas

***1. CLI Not Found -- Install with `brew install tina4stack/tap/tina4`.***

***2. Wrong Language Detected -- Ensure `package.json` exists.***

***3. Generate Overwrites Existing Files -- Check before generating.***

***4. Migration Already Applied -- Create new migrations for changes.***

***5. Custom Command Not Found -- Place in `src/commands/`.***

***6. Build Fails -- Check TypeScript errors with `npx tsc --noEmit`.***

***7. Port Conflict -- Use `tina4 serve --port 8080`.***

# Chapter 20: Deployment

## 1. From Development to Production

You have built the app. It runs on `localhost:7148`. Now it needs to run 24/7. Handle thousands of concurrent users. Survive server restarts. Not leak memory.

This chapter covers everything for production deployment.

## 2. Production .env Configuration

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7148

# Database
DATABASE_URL=sqlite:///data/app.db

# Security
CORS_ORIGINS=https://yourdomain.com
TINA4_JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_CACHE_TEMPLATES=true
TINA4_MINIFY_HTML=true

# Console (disabled)
TINA4_CONSOLE=false
```

### *Key Differences*

| Setting | Dev | Production |

|---------|-----|------------|

| `TINA4_DEBUG` | `true` | `false` |

| `TINA4_LOG_LEVEL` | `ALL` | `WARNING` |

| `CORS_ORIGINS` | `*` | Your domain |

| `TINA4_CONSOLE` | `true` | `false` |

## 3. Building for Production

```
tina4 build
```

```
Building for production...
  Compiled 47 TypeScript files
  Output: dist/
  Size: 245KB
```

Run in production:

```
node dist/app.js
```

## 4. Docker Deployment

Create `Dockerfile`:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY dist/ ./dist/
COPY src/templates/ ./src/templates/
COPY src/public/ ./src/public/
COPY src/migrations/ ./src/migrations/

ENV TINA4_DEBUG=false
ENV TINA4_PORT=7148

EXPOSE 7148

CMD ["node", "dist/app.js"]
```

Create `docker-compose.yml`:

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7148:7148"
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    env_file:
      - .env.production
    restart: unless-stopped

volumes:
  app-data:
  app-logs:
```

Build and run:

```
tina4 build
docker compose up -d
```

## 5. Node.js Cluster for Production

For multi-core utilization, Tina4 supports Node.js cluster mode:

```
TINA4_CLUSTER=true
TINA4_CLUSTER_WORKERS=4
```

Or set workers to `auto` to match CPU cores:

```
TINA4_CLUSTER_WORKERS=auto
```

This spawns multiple worker processes. Each handles requests independently. A worker crashes. The cluster master respawns it. No downtime.

## 6. Nginx Reverse Proxy

```
server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:7148;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ws/ {
        proxy_pass http://127.0.0.1:7148;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
// ...
```

## 7. SSL with Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com
```

## 8. Process Management with systemd

Create `/etc/systemd/system/tina4-app.service`:

```
[Unit]
Description=Tina4 Node.js Application
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/my-app
ExecStart=/usr/bin/node /var/www/my-app/dist/app.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production
EnvironmentFile=/var/www/my-app/.env.production

[Install]
```

```
WantedBy=multi-user.target
```

```
sudo systemctl enable tina4-app
sudo systemctl start tina4-app
sudo systemctl status tina4-app
```

## 9. Queue Workers in Production

```
[Unit]
Description=Tina4 Queue Worker
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/my-app
ExecStart=/usr/local/bin/tina4 queue:work
Restart=always
RestartSec=5
EnvironmentFile=/var/www/my-app/.env.production

[Install]
WantedBy=multi-user.target
```

## 10. Health Check and Monitoring

```
curl http://localhost:7148/health
```

### *Broken File Check*

Tina4 watches for a `.broken` file in production. When the file exists, the health check returns `503`. A signal to the load balancer: stop sending traffic.

```
touch .broken          # Health check returns 503
# Deploy new code...
rm .broken             # Health check returns 200
```

## 11. Zero-Downtime Deployment

```
#!/bin/bash
set -e

cd /var/www/my-app

# Signal load balancer to stop sending traffic
touch .broken
sleep 5

# Pull latest code
git pull origin main

# Install dependencies and build
npm ci --only=production
tina4 build

# Run migrations
tina4 migrate
```

```
# Restart the service
sudo systemctl restart tina4-app

# Wait for health check
sleep 2
curl -f http://localhost:7148/health
// ...
```

## 12. Scaling

### *Vertical Scaling*

Use cluster mode with more workers:

```
TINA4_CLUSTER=true
TINA4_CLUSTER_WORKERS=8
```

### *Horizontal Scaling*

Run multiple instances behind a load balancer. Use Redis for shared sessions, cache, and queues:

```
TINA4_SESSION_HANDLER=redis
TINA4_CACHE_BACKEND=redis
TINA4_QUEUE_BACKEND=rabbitmq
```

## 13. Exercise: Deploy TaskFlow to Docker

Dockerize the TaskFlow app from Chapter 21 with Nginx, SSL, and a queue worker.

## 14. Solution

```
tina4 build
docker compose -f docker-compose.prod.yml up -d
```

## 15. Gotchas

***1. Debug Mode in Production -- Set `TINA4_DEBUG=false`.***

***2. .env in Version Control -- Add to `.gitignore`.***

***3. SQLite in Multi-Instance -- Use PostgreSQL for horizontal scaling.***

***4. WebSocket Behind Load Balancer -- Configure sticky sessions or use Redis pub/sub.***

***5. Memory Leaks -- Monitor with `process.memoryUsage()` and restart workers periodically.***

***6. Missing Migrations -- Always run `tina4 migrate` during deployment.***

***7. Port Conflicts -- Ensure only one process listens on port 7148.***

# Chapter 21: Building a Complete App

## 1. Putting It All Together

Twenty chapters. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment.

Time to use all of it. From scratch.

We are building **TaskFlow** -- a task management system with:

• User registration and JWT authentication

• Task creation, assignment, and tracking

• A dashboard with real-time updates

• Email notifications when tasks are assigned

• Caching for dashboard performance

• A full test suite

• Docker deployment

## 2. Planning the App

### *Models*

| Model | Table | Fields |

|-------|-------|--------|

| User | users | id, name, email, passwordHash, role, createdAt |

| Task | tasks | id, title, description, status, priority, createdBy, assignedTo, dueDate, completedAt, createdAt, updatedAt |

### *Relationships*

• A User has many Tasks (created by them)

• A User has many Tasks (assigned to them)

• A Task belongs to a User (creator)

• A Task belongs to a User (assignee)

### *Routes*

| Method | Path | Description | Auth |

|--------|------|-------------|------|

| POST | /api/auth/register | Register a new user | public |

| POST | /api/auth/login | Login, get JWT | public |

| GET | /api/profile | Get current user profile | secured |

| GET | /api/tasks | List tasks (with filters) | secured |

| GET | /api/tasks/:id | Get a single task | secured |

| POST | /api/tasks | Create a task | secured |

| PUT | /api/tasks/:id | Update a task | secured |

| DELETE | /api/tasks/:id | Delete a task | secured |

| GET | /api/dashboard/stats | Dashboard statistics | secured |

| GET | /admin | Dashboard HTML page | public |

## 3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow
cd taskflow
npm install
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=dev-token
TINA4_JWT_SECRET=taskflow-dev-secret-change-in-production
TINA4_JWT_EXPIRY=86400
```

### *Create Migrations*

Create `src/migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
```

```
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

```
tina4 migrate
```

## 4. Step 2: Define Models

Create `src/orm/User.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class User extends BaseModel {
    static tableName = "users";
    static primaryKey = "id";
    static hasMany = [
        { model: "Task", foreignKey: "created_by", as: "createdTasks" },
        { model: "Task", foreignKey: "assigned_to", as: "assignedTasks" }
    ];

    id!: number;
    name!: string;
    email!: string;
    passwordHash!: string;
    role: string = "user";
    createdAt!: string;
}
```

Create `src/orm/Task.ts`:

```
import { BaseModel } from "tina4-nodejs";

export class Task extends BaseModel {
    static tableName = "tasks";
    static primaryKey = "id";
    static belongsTo = [
        { model: "User", foreignKey: "created_by", as: "creator" },
        { model: "User", foreignKey: "assigned_to", as: "assignee" }
    ];

    id!: number;
    title!: string;
    description: string = "";
    status: string = "todo";
    priority: string = "medium";
    createdBy!: number;
    assignedTo: number | null = null;
    dueDate: string | null = null;
    completedAt: string | null = null;
    createdAt!: string;
    updatedAt!: string;
}
```

## 5. Step 3: Auth Middleware

Create `src/routes/middleware.ts`:

```
import { Auth } from "tina4-nodejs";

export async function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        return res.status(401).json({ error: "Authorization required" });
    }

    const token = authHeader.substring(7);

    if (!Auth.validToken(token)) {
        return res.status(401).json({ error: "Invalid or expired token" });
    }

    req.user = Auth.getPayload(token);
    return next(req, res);
}
```

## 6. Step 4: Auth Routes

Create `src/routes/auth.ts`:

```
import { Router, Auth, Database } from "tina4-nodejs";

/**
 * Register a new user
 * @noauth
 * @tags Auth
 * @body {"name": "string", "email": "string", "password": "string"}
 * @response 201 {"message": "string", "user": {"id": "int", "name": "string", "email": "string"}}
 */
Router.post("/api/auth/register", async (req, res) => {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
        return res.status(400).json({ error: "Name, email, and password are required" });
    }

    if (password.length < 8) {
        return res.status(400).json({ error: "Password must be at least 8 characters" });
    }

    const db = Database.getConnection();
    const existing = await db.fetchOne("SELECT id FROM users WHERE email = :email", { email });

    if (existing) {
        return res.status(409).json({ error: "Email already registered" });
// ...
```

## 7. Step 5: Task Routes

Create `src/routes/tasks.ts`:

```
import { Router, Database, Queue } from "tina4-nodejs";
```

```
/**
 * List tasks with filters
 * @tags Tasks
 * @query string status Filter by status (todo, in_progress, done)
 * @query string priority Filter by priority (low, medium, high)
 * @query string assigned Filter by assigned user ID
 * @query int page Page number
 */
Router.get("/api/tasks", async (req, res) => {
    const db = Database.getConnection();
    const userId = req.user.user_id;

    const conditions = ["(created_by = :userId OR assigned_to = :userId)"];
    const params: Record<string, any> = { userId };

    if (req.query.status) {
        conditions.push("status = :status");
        params.status = req.query.status;
    }

    if (req.query.priority) {
        conditions.push("priority = :priority");
        params.priority = req.query.priority;
// ...
```

## 8. Step 6: Dashboard Stats

Create `src/routes/dashboard.ts`:

```
import { Router, Database, cacheGet, cacheSet } from "tina4-nodejs";

Router.get("/api/dashboard/stats", async (req, res) => {
    const userId = req.user.user_id;
    const cacheKey = `dashboard:${userId}`;

    const cached = await cacheGet(cacheKey);
    if (cached) {
        return res.json({ ...cached, source: "cache" });
    }

    const db = Database.getConnection();

    const total = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE created_by = :userId OR assigned_to = :userId",
        { userId }
    );

    const byStatus = await db.fetch(
        "SELECT status, COUNT(*) as count FROM tasks WHERE created_by = :userId OR assigned_to = :userId GROU
        { userId }
    );

    const overdue = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE (created_by = :userId OR assigned_to = :userId) AND status
// ...
```

## 9. Step 7: WebSocket for Real-Time Updates

Create `src/routes/ws-tasks.ts`:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/tasks", async (connection, event, data) => {
    if (event === "open") {
        connection.send(JSON.stringify({ type: "connected", message: "Listening for task updates" }));
    }
});
```

## 10. Step 8: Queue Consumer for Notifications

Create `src/routes/queue-consumers.ts`:

```
import { Queue, Database, Messenger, Frond } from "tina4-nodejs";

Queue.consume("task-notifications", async (job) => {
    const { type, task_title, assigned_to, assigned_by } = job.payload;

    const db = Database.getConnection();
    const user = await db.fetchOne("SELECT name, email FROM users WHERE id = :id", { id: assigned_to });

    if (!user) return true;

    console.log(`[Notification] Task "${task_title}" assigned to ${user.name} by ${assigned_by}`);

    const mailer = new Messenger();
    await mailer.send(
        user.email,
        `New Task Assigned: ${task_title}`,
        `<h2>New Task Assigned</h2><p>Hi ${user.name},</p><p>${assigned_by} assigned you a new task: <strong>S
    );

    return true;
});
```

## 11. Step 9: Tests

Create `tests/TaskFlowTest.ts`:

```
import { Test, Auth } from "tina4-nodejs";

export class TaskFlowTest extends Test {
    private token: string = "";
    private taskId: number = 0;

    async setupOnce() {
        await this.testPost("/api/auth/register", {
            name: "Test User",
            email: "test@taskflow.com",
            password: "securePass123"
        });

        const login = await this.testPost("/api/auth/login", {
            email: "test@taskflow.com",
            password: "securePass123"
        });

        this.token = login.body.token;
    }
```

```
    async testCreateTask() {
        const response = await this.testPost("/api/tasks", {
            title: "Write tests",
            description: "Add comprehensive test coverage",
// ...
```

Run tests:

```
npm test
```

## 12. Step 10: Docker Deployment

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY dist/ ./dist/
COPY src/templates/ ./src/templates/
COPY src/public/ ./src/public/
COPY src/migrations/ ./src/migrations/
ENV TINA4_DEBUG=false
EXPOSE 7148
CMD ["node", "dist/app.js"]
```

```
tina4 build
docker build -t taskflow .
docker run -p 7148:7148 -v taskflow-data:/app/data taskflow
```

## 13. What We Built

TaskFlow exercises every major Tina4 feature:

• **Routing** -- RESTful API with explicit route registration

• **ORM** -- User and Task models with relationships

• **Database** -- SQLite with migrations, parameterised queries

• **Authentication** -- JWT tokens, password hashing, auth middleware

• **Middleware** -- Auth protection on route groups

• **Queues** -- Background email notifications

• **WebSocket** -- Real-time task updates

• **Caching** -- Dashboard stats cached for 30 seconds

• **Templates** -- Dashboard HTML page with tina4css

• **Swagger** -- Annotated API documentation

• **Testing** -- Full test suite

• **Deployment** -- Docker with Nginx

All of this in a single npm package. Zero dependencies. One framework doing the work of twelve.

# Chapter 22: Vibe Coding with AI

## Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to know which packages to install, which config files to create, which naming conventions to follow, and how to wire everything together. It hallucinates half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first try because there is only one way to do things in Tina4.

This is not accidental. Tina4 was built from the ground up as the best framework for AI-assisted development.

## The Zero-Dependency Advantage

When an AI generates code for Express, it might suggest `express-session` or `cookie-session` or `connect-redis`. All exist. All work differently. The AI guesses.

With Tina4, there is one cache. One queue. One ORM. One template engine. No ambiguity. No alternatives. No "it depends on which package you installed."

```
// There is only one way to cache in Tina4
const data = await cacheGet("products");

// There is only one way to queue in Tina4
await Queue.produce("emails", { to: "user@test.com" });

// There is only one way to send email in Tina4
const mail = new Messenger();
await mail.send("user@test.com", "Welcome", "<h1>Welcome!</h1>");
```

Zero dependencies means zero confusion for AI.

## CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works:

• Every method signature with parameters and return types

• The project structure convention (`src/routes/`, `src/orm/`, `src/templates/`)

• The `.env` variable reference

• Code style rules (routes use `async (req, res) =>`, ORM uses `toDict()`)

• Database connection string formats

• Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

## Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

### *tina4-maintainer*

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

### *tina4-developer*

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

### *tina4-js*

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

## The Convention Advantage

AI thrives on convention. Every Tina4 project follows the same structure. The AI never asks "where should I put this?"

```
src/
  routes/hello.ts       → AI knows: this is a route file
  orm/Product.ts        → AI knows: this is an ORM model
  templates/home.twig   → AI knows: this is a template
```

Tell the AI "add a User model" and it creates `src/orm/User.ts` with the correct class definition, the correct imports, and the correct property types. Every time.

## Real-World Vibe Coding Session

Here is an actual conversation with Claude Code in a Tina4 Node.js project:

**You:** "Add a blog with posts and comments. Posts belong to users. Comments belong to posts. I want a REST API and a template page that lists posts."

**Claude Code generates:**

1. `src/orm/Post.ts` -- Model with belongsTo User, hasMany Comments
2. `src/orm/Comment.ts` -- Model with belongsTo Post
3. `src/routes/api/posts.ts` -- Full CRUD API
4. `src/routes/api/comments.ts` -- Nested comments API
5. `src/routes/blog.ts` -- Template route for the blog page
6. `src/templates/blog.twig` -- Page with tina4css cards for each post
7. `src/migrations/20260322_create_posts_and_comments.sql` -- Database schema

All correct. All runnable. First try.

**You:** "Add pagination to the posts list"

**Claude adds** `req.query.page` handling and proper offset calculation -- because CLAUDE.md told it exactly how Tina4 pagination works.

**You:** "Cache the post list for 5 minutes"

**Claude adds** `cacheGet("posts_page_" + page)` with `cacheSet("posts_page_" + page, data, 300)` -- because it knows the cache API.

**You:** "Send an email when a new comment is posted"

**Claude adds** `const mail = new Messenger(); await mail.send(...)` -- because it knows Messenger reads from `.env`.

No research. No Stack Overflow. No "which package should I use?" Describe what you want. The AI builds it.

## TypeScript-Specific Advantages

TypeScript makes AI coding even more reliable:

```
// The AI knows the exact types
import { Router, Database, Auth, Queue, Messenger, cacheGet, cacheSet } from "tina4-nodejs";
import { BaseModel } from "tina4-nodejs";

// Type declarations guide the AI
export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";
    static hasMany = [{ model: "Review", foreignKey: "product_id" }];

    id!: number;
    name!: string;
    price: number = 0;
    inStock: boolean = true;
}

// The AI generates type-safe route handlers
Router.get("/api/products/:id:int", async (req, res) => {
    const product = new Product();
    await product.load(req.params.id);
    return product.id ? res.json(product.toDict()) : res.status(404).json({ error: "Not found" });
});
```

TypeScript's type system is a guardrail. It prevents the AI from generating code that fails at runtime. Property types, method signatures, and return types steer the AI toward correct code.

## Supported AI Tools

Tina4 auto-detects and installs context for:

| Tool | Detection | Context installed |

|------|----------|----------|

| Claude Code | `.claude/` directory | CLAUDE.md + skills |

| Cursor | `.cursor/` directory | .cursorrules |

| GitHub Copilot | `.github/copilot/` | instructions.md |

| Windsurf | `.windsurfrules` file | .windsurfrules |

| Aider | `.aider.conf.yml` | .aider.conf.yml |

| Cline | `.cline/` directory | .clinerules |

| OpenAI Codex | `.codex/` directory | instructions.md |

Run `tina4 doctor` to see which tools are detected.

## The AI Chat in Dev Dashboard

The dev dashboard at `/tina4/console` includes an AI chat tab. Enter your API key. Chat with Claude or GPT about your code directly from the browser.

The AI has full context of your Tina4 project. Ask it anything:

• "Why is my /api/users route returning 500?"

• "How do I add WebSocket support?"

• "Write a migration to add an avatar column to users"

## Tips for Effective Vibe Coding with Tina4

1. **Be specific about what you want** -- "Add a product API with search by name and price range" works better than "add products"

2. **Let the AI use tina4 generate** -- "Run tina4 generate model Product then add price and category fields"

3. **Reference the .env** -- "Configure the queue to use RabbitMQ" -- the AI knows to update .env

4. **Ask for tests** -- "Write tests for the Product API" -- the AI uses Tina4's inline testing

5. **Review, don't rewrite** -- The AI's first attempt is usually 90% correct. Tweak the details.

## Exercise

Open your Tina4 Node.js project in Claude Code (or your preferred AI tool) and try these prompts:

1. "Add a Contact model with name, email, phone, and message fields"

2. "Create a contact form page at /contact with a Twig template using tina4css"

3. "Add an API endpoint that accepts the form submission and saves it to the database"

4. "Send an email notification when a new contact is submitted"

5. "Add rate limiting to the contact form -- max 3 submissions per minute"

Each prompt should generate correct, runnable TypeScript code on the first try.

## Gotchas

1. **No CLAUDE.md?** Run `tina4 init` or `tina4 doctor` to generate it

2. **AI suggests wrong import paths?** The CLAUDE.md might be outdated -- regenerate it

3. **AI hallucinates a package?** Remind it: "Tina4 has zero dependencies, use only built-in features"

4. **AI creates files in wrong location?** Tell it: "Follow the src/routes, src/orm, src/templates convention"

5. **AI code doesn't run?** Check that routes return `res.json()` not just `console.log()` -- Tina4 convention matters

6. **AI uses Express patterns?** Remind it: "This is Tina4, not Express. Import from tina4-nodejs."

7. **AI forgets await?** All Tina4 database and ORM methods are async and need await

## The Philosophy

Tina4 was not retrofitted for AI coding tools. It was designed for them from the ground up.

Convention over configuration means the AI knows where things go. Zero dependencies means the AI never chooses between packages. A single CLAUDE.md file means the AI has complete framework knowledge. Identical APIs across 4 languages means that knowledge transfers instantly.

You describe the intent. The AI writes the code. You review and refine. Tina4 is the ground the partnership stands on.

*This is not a framework. This is a partnership.*